Vietnam General Confederation of Labor

**Ton Duc Thang University**

**Faculty of Information Technology**


**Bui Phuong Nam – 522H0002**

**Hoang Minh Quan – 523002451**

# A Comparison of Monolithic, Modular Monolith, and Microservices Architectures in LMS Systems

INFORMATION TECHNOLOGY PROJECTS

**SOFTWARE ENGINEERING**

Supervisor

**Msc. Mai Van Manh**

**Ho Chi Minh City, 2026**

# Acknowledgment

First and foremost, I would like to express my sincere gratitude to my supervisors, **MSc. Mai Van Manh**, for his invaluable guidance, constructive feedback, and continuous encouragement throughout the duration of this thesis. His expertise, patience, and dedication have played a crucial role in shaping both the technical direction and academic rigor of this research.

I am also deeply thankful to the lecturers and staff of the Faculty of Information Technology at Ton Duc Thang University, whose support and resources provided a strong foundation for my research journey. Their commitment to creating an inspiring and intellectually stimulating environment has greatly contributed to my growth.

My gratitude further extends to my colleagues, peers, and friends who offered helpful discussions, motivation, and meaningful insights during the development and evaluation of this project. Their companionship made the challenging moments far more manageable.

Finally, I am profoundly grateful to my family for their unconditional support, patience, and encouragement. Their belief in me has always been my greatest source of strength and determination.

To all of you, I extend my heartfelt thanks.

# Certification

This Projects was carried out at **Ton Duc Thang University**.

**Advisor: MSc. Mai Van Manh**

.....

*(Title, Full Name and Signature)*

This project was defended at the Undergraduate Thesis Examination Committee held at Ton Duc Thang University on . . . . . / . . . . . / . . . . . .

Confirmation of the Chairman of the Undergraduate Thesis Examination Committee and the Dean of the Faculty after receiving the revised thesis (if any).

| **CHAIRMAN** | **DEAN OF FACULTY** |
| --- | --- |
| . . . . . | . . . . . |

# Declaration

This project was completed at **Ton Duc Thang University**. We hereby declare that this is our own research work, conducted under the academic supervision of **MSc. Mai Van Manh**. All analyses, discussions, and results presented in this project are truthful, original, and have not been published in any form prior to this submission.

All data used for analysis, evaluation, and comparison were collected directly by the author(s) from various reliable sources, which are fully cited in the reference section of this project. In addition, certain evaluations, comments, and figures quoted from other authors, organizations, or research works are clearly referenced and properly acknowledged.

If any form of academic dishonesty, plagiarism, or copyright infringement is discovered, we accept full responsibility for the content of this project. Ton Duc Thang University shall bear no liability for any copyright or authorship violations committed by the author(s) during the execution of this research.

Ho Chi Minh City, day month year

**Author(s)**

*(Signature and full name)*

# Abstract

The selection of an appropriate software architecture plays a critical role in the development and evolution of **Learning Management Systems (LMS)** , directly affecting system scalability, maintainability, and adaptability to changing requirements. This project presents a comparative study of three widely adopted architectural approaches—**Monolithic**, **Modular Monolith** , and **Microservices**—within the context of an LMS application.

A core LMS system was designed with essential functionalities, including user management, course management, enrollment, learning content delivery, and assessment handling. Based on the same functional requirements, the system was implemented separately using the three architectural styles to ensure a fair and consistent comparison. Several change and extension scenarios, such as modifying grading logic, adding learning progress statistics, and preparing for future learning analytics integration, were defined and applied to each architecture.

The architectures were evaluated using technical criteria including module isolation, change impact scope, scalability at the functional level, deployment and operational complexity, and suitability for different system scales. The experimental results highlight the strengths and limitations of each approach: Monolithic architecture offers simplicity and ease of deployment but exhibits limited flexibility when evolving; Modular Monolith improves maintainability through clearer module boundaries while retaining centralized deployment; Microservices provide superior scalability and change isolation at the cost of increased implementation and operational complexity.

This study provides practical insights into the real-world trade-offs of architectural choices and proposes recommendations for selecting suitable architectures for LMS systems of small, medium, and large scales.

# Contents

# List of Figures

# List of Tables

# Chapter 1

# Introduction

## 1.1 Background and Rationale

In recent years, the rapid growth of technology and the push for digital transformation in education have made the development of robust Learning Management Systems (LMS) more urgent than ever. The architecture behind such systems is not merely a technical choice; it directly shapes their scalability, maintainability, and long-term. Approaches such as Monolithic and Microservices have emerged as dominant paradigms, each offering unique strengths and trade-offs. This project will explore and contrast these three architectural models in the context of building an LMS, focusing on functional modules such as user authentication, course management, content delivery, assessment, and progress tracking.

## 1.2 Project Objectives

In general , the objectives of this project is building and deploying a Learning Management System (LMS) in form of three different software architectures.

For specific, the following targets will be expected:

1. Design an LMS system with core functions (CRUD, authentication / authorization, enrollment, learning content, assignment & grade, learning process,..)
2. Deploy the LMS system according to three different architectures
3. Develop scenarios for system change and expansion
4. Compare architectures based on specific technical criteria
5. Analyze the advantages and disadvantages of each architecture in actual implementation

## 1.3 Scope of the Project

The scope of the project focuses on researching, designing and comparing three software architectures when applied to the same business problem. The LMS system is only used as a research object to illustrate and verify the above architectures, not the main development goal of the project. In other words, the focus of the topic is software architecture analysis, while LMS construction only serves as a means for deployment and evaluation.

Across all three architectures, the system remains core functional modules stable (e.g., authentication and authorization, enrollment, learning content, assignments and grading, progress tracking) even existing changes in architecture. Extended features such as learning analytics or recommendation algorithms are acknowledged but reserved for future work.

The project does not prioritize detailed user interface or user experience (UI/UX) design, employing only minimal interfaces sufficient for testing and demonstration. Regarding technology choices, the team adopts contemporary frameworks considered most suitable at present, namely NestJS and Vue.js due to their widespread adoption, strong community support, and suitability for modular.

Other architectures (e.g., serverless, large-scale event-driven architecture, or complex hybrid architectures) are outside the scope of the study.

## 1.4 Research Methodology

With a comparative research orientation, this project focuses on an empirical architectural study rather than a pure software development process.

### 1.4.1 Theorical Methodology

The first phase includes refering and reviewing design principle theories to clarify the basic characteristics of the elearning system and building techniques. Those theories are collected from reputable academic papers and professional blogs [**?**,**?**]. This phases also need to figure out the advantages and limitations of the three architectures. During refering, our team proposes some search keywords including: “monolithic and microservices”, ”elearning systems design”, ”Low & High coupling”.

The theories analysis process is carried out using a thematic synthesis approach, focusing on extracting core principles such as coupling, cohesion, deployment unit, inter-component communication, and scalability. From there, the study identifies five main comparison criteria:

- Degree of module separation,
- Scope of impact when modifying functionality,
- Scalability of individual functional components,
- Complexity of deployment and operation,
- Suitability for different system scales: small, medium, large.

The result of the theoretical phase is a comparison framework that serves as the foundation for the experimental part, ensuring that the assessment is not subjective but based on recognized principles and evidence.

### 1.4.2 Practical Methodology

The experimental phase is the focus of the research. At the begining, team will outline some ideas and functional requirement into Software Requirement Specs (SRS) in order to be convenient for designing. In SRS, the system is decomposed into consistent business subsystems including: Identity & Access Management, User Profile, Course Management, Enrollment Management, Learning Content, Assessment & Grading, Progress Tracking, along with future expansion subsystems (Learning Analytics and AI). Maintaining the same business

decomposition across all three architectures ensures fairness and objectivity: differences only come from the choice of architectural structure, not from functional variation.

The LMS system is implemented three times, each corresponding to one of the selected architectural styles. All of them, we use the same technology stack (NestJS, Vuejs, PostgreSQL) to control extraneous variables, and are containerized with Docker to measure deployments fairly.

A scenario-based evaluation approach is applied to all implementations using predefined change scenarios, including:

- Modifying the learning scoring logic
- Adding a learning progress statistics feature
- Preparing for future integration of learning analytics functionality

The evaluation is performed using scenario-based evaluation, a common approach in software architecture research to simulate real-life changes. Three change scenarios are applied to all three versions in turn:

- Modifying the learning scoring logic.
- Adding a learning progress statistics feature (adding a learning progress statistics feature).
- Preparing for future integration of learning analytics functionality.

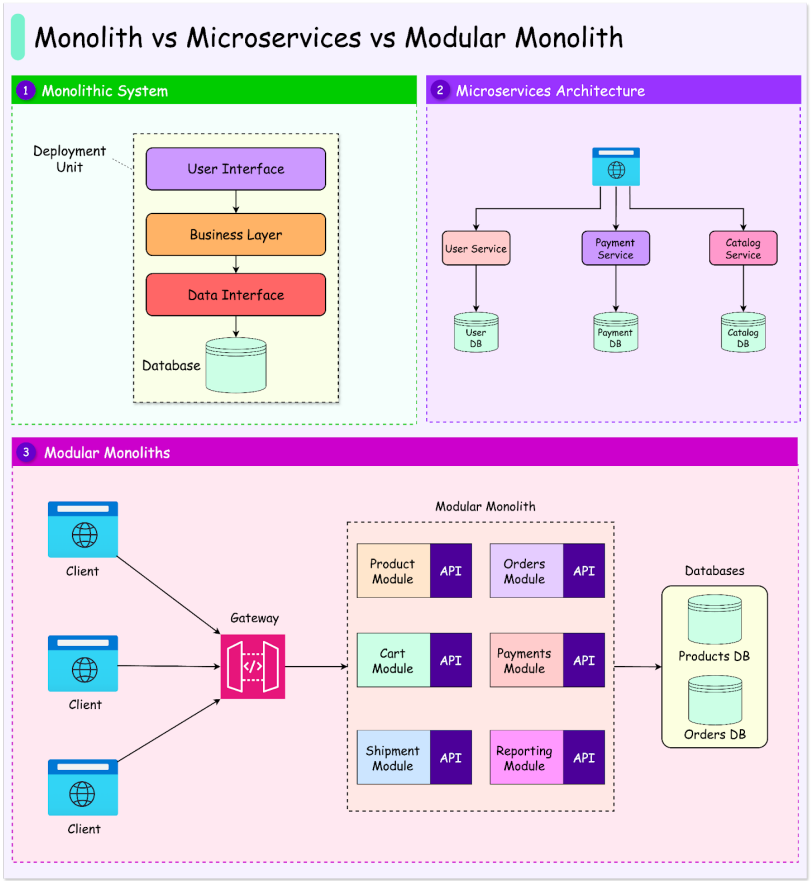



Figure 1.1: Monolith vs Microservices: What's the Right Choice [?]

## 1.5 Structure of the Report

This report is structured as follows:

- **Chapter 1**: Introduction - Provides background, objectives, scope, methodology, and references.
- **Chapter 2**: Literature Review - Reviews existing technologies and related work.
- **Chapter 3**: System Design and Implementation - Details the design and implementation of the messaging system.
- **Chapter 4**: Testing and Evaluation - Presents the testing procedures and evaluation results.
- **Chapter 5**: Conclusion and Future Work - Summarizes findings and suggests future directions.
- **Appendices**: Contains supplementary material and code listings.

# Chapter 2

# Literature Review

This chapter provides a comprehensive review of existing literature, relevant technologies, and theoretical foundations that underpin the design and evaluation of software architectures in the context of Learning Management Systems (LMS). By synthesizing insights from prior research and established architectural principles, this chapter establishes the conceptual framework used in subsequent experimental chapters.

This chapter provides a comprehensive review of existing literature, relevant technologies, and theoretical foundations that underpin the design and evaluation of software architectures in the context of Learning Management Systems (LMS). By synthesizing insights from prior research and established architectural principles, this chapter establishes the conceptual framework used in subsequent experimental chapters.

## 2.1 Related Works

The relationship between software architecture selection and system quality has been a recurring subject in both academia and industry. A growing body of work examines how architectural choices affect maintainability, scalability, and the cost of change — the three dimensions most relevant to this study. ========

### 2.1.1 Existing Learning Management System Implementations

Learning Management Systems have traditionally been implemented as monolithic applications due to their relative simplicity and low initial infrastructure cost. Early open-source platforms such as Moodle and Blackboard exemplify this approach, bundling authentication, course management, content delivery, and assessment into a single deployable unit. While this architectural choice accelerates initial development, it introduces well-documented challenges as the system matures: tightly coupled modules hinder independent deployment, and a fault in one subsystem can cascade across the entire platform.

More recent commercial LMS platforms, notably Canvas and Google Classroom, have progressively migrated toward service-oriented. These migrations, often documented as engineering case studies, highlight a common pattern: monolithic systems accumulate technical debt at a rate proportional to team size and feature velocity, eventually motivating architectural refactoring. Dinh (2023) describes the design and implementation of an e-learning system and identifies modular decomposition as a prerequisite for long-term maintainability [**?**]. This observation is further reinforced by Tapia et al. (2020), who argue that the traditional monolithic architecture no longer meets the demands of modern scalable systems. [**?**]. Nevertheless, Tapia themselves acknowledge that a monolithic codebase may be more efficient and carry less overhead for smaller-scale applications, and

that migrating to a microservices design does not guarantee benefits for all organizations. This view is echoed by Niemelä and Hyyrö (2019), who caution that microservice architecture will provide neither a single point of failure nor easy logging, and that service orchestration is in fact quite complex, with poorly informed changes potentially causing discontinuities in service provision [**?**]. Taken together, these perspectives suggest that the choice between monolithic and microservice architecture should be driven by the specific scale, complexity, and operational context of the system under development, rather than by architectural fashion alone.

### 2.1.2 Comparative Studies on Software Architectures

The comparative evaluation of Monolithic and Microservices architectures has attracted significant research attention. Xu (2025) provides a widely referenced comparative overview, arguing that each architectural style occupies a distinct position on the spectrum between simplicity and operational flexibility [**?**]. The study concludes that no single architecture universally outperforms the others; rather, the appropriate choice is contingent on team size, system scale, and expected rate of change.

Empirical studies applying scenario-based evaluation — a methodology where predefined change scenarios are applied to each architectural variant — have demonstrated measurable differences in change impact scope. Research in enterprise software contexts consistently reports that Microservices architectures isolate changes to smaller code surfaces but introduce higher coordination costs through inter-service communication.

The gap in the existing literature that this project addresses is the scarcity of studies that implement all three architectures on a single, well-defined application domain under identical functional requirements, then apply consistent change scenarios across all three variants. Most comparative studies rely on theoretical analysis or survey data rather than controlled empirical implementation.

## 2.2 Theoretical Principles and Architectural Models

The comparative framework applied in this study is grounded in established theoretical principles from software architecture research. This section synthesizes the relevant principles and defines the conceptual vocabulary used in the experimental evaluation.

### 2.2.1 Monolithic Architecture

A Monolithic architecture deploys all application functionality as a single executable unit. The entire application — encompassing presentation logic, business logic, and data access — is compiled and deployed together. Communication between functional areas occurs through direct function calls within the same process memory space, eliminating network latency for internal interactions. The primary strengths of Monolithic architectures are development simplicity and deployment straightforwardness. A development team can run the entire system locally with a single command, and deployment requires only a single artifact. These properties make Monolithic architectures well-suited for small teams and early-stage projects where time-to-market and operational simplicity take precedence. The documented limitations of Monolithic architectures become apparent as systems grow. Because all modules share a single deployment unit, a change to any module necessitates redeployment of the entire application. Tight coupling between modules — particularly when modules access each other's data structures directly — means that changes propagate unpredictably through the codebase. Scaling is possible only at the application level; it is not feasible to scale a computationally intensive module independently of the rest of the system.

### 2.2.2 Microservices Architecture

Microservices architecture decomposes an application into a collection of small, independently deployable services, each responsible for a distinct business capability. Services communicate through lightweight protocols, typically HTTP REST or asynchronous message queues, and are deployed and scaled independently. The defining characteristics of Microservices architectures include: independent deployability, enabling a team to release a change to one service without coordinating with other teams; polyglot persistence, allowing each service to select the database technology most appropriate for its workload; fault isolation, limiting the blast radius of a service failure to that service's consumers; and independent scalability, enabling resource allocation proportional to the demand on each service. These properties come at a cost. The introduction of network boundaries between services introduces latency and failure modes absent in single-process architectures. Distributed transactions — operations requiring consistency across multiple services — require explicit coordination patterns such as sagas or two-phase commit. Operational complexity increases significantly: the system requires service discovery, load balancing, distributed tracing, centralized logging, and health monitoring infrastructure that is unnecessary in Monolithic deployments. For LMS systems of small to medium scale, these overheads may outweigh the scalability and isolation benefits.

### 2.2.3 Coupling, Cohesion, and Change Impact

Theoretical Foundations The concepts of coupling and cohesion were formally established by Yourdon and Constantine (1979) in Structured Design: Fundamentals of a Discipline of Computer Program and Systems Design, where they were organized into a classification hierarchy ranging from the most desirable to the least desirable forms of each property. Their classification of coupling spans from data coupling — in which modules share only the minimum necessary data — through stamp, control, external, and common coupling, to content coupling, in which one module directly modifies the internals of another. Cohesion is similarly ordered from coincidental cohesion, in which elements of a module are grouped arbitrarily, to functional cohesion, in which every element contributes to a single, well-defined purpose. The theoretical justification for module decomposition based on these properties was articulated by Parnas (1972) in the seminal paper "On the Criteria To Be Used in Decomposing Systems into Modules." Parnas argued that the correct criterion for decomposition is the principle of information hiding: each module should encapsulate a design decision that is likely to change, exposing only what is necessary through a stable interface. This principle directly informs the module boundary definitions applied in Microservices implementations in this study.

Measuring Coupling Robert C. Martin (2000) extended these foundational concepts into quantifiable metrics applicable to object-oriented and component-based systems. Two complementary coupling measures are defined at the module level. Afferent coupling (Ca) counts the number of external modules that depend on a given module — a measure of its responsibility to others. Efferent coupling (Ce) counts the number of external modules upon which a given module depends — a measure of its sensitivity to external change. From these, Martin derives the Instability metric:

$$I = \frac{C_e}{C_a + C_e}, \quad I \in [0, 1] \tag{2.1}$$

A module with I = 0 is maximally stable: many components depend on it, but it depends on nothing external, making it resistant to change propagation. A module with I = 1 is maximally unstable: it depends heavily on external components while nothing depends on it. A well-designed architecture positions core business logic modules at low instability values and infrastructure or adapter modules at high instability values, reflecting the

Stable Dependencies Principle. In the context of this study, coupling is operationalized through two measurable proxies: the number of cross-module import statements identified through static analysis of each implementation, and the number of modules requiring modification in response to each defined change scenario.

### 2.2.4 Measuring Cohesion

At the class level, the Chidamber and Kemerer (1994) metric suite provides a widely adopted measure of cohesion through the Lack of Cohesion of Methods (LCOM). In its standard formulation, LCOM is defined as:

$$LCOM = \begin{cases} |P| - |Q| & \text{if } |P| > |Q| \\ 0 & \text{otherwise} \end{cases}$$

where P is the set of method pairs that share no common instance attributes, and Q is the set of method pairs that share at least one common instance attribute. A high LCOM value indicates that a class addresses multiple unrelated concerns and would benefit from decomposition. While LCOM is defined at the class level, the underlying principle generalizes to the module and service level: a module exhibiting high cohesion groups classes and functions that collaborate around a single bounded context, with minimal dependence on the internals of other modules. In this study, cohesion is assessed at the module level through the proportion of classes within each module that belong to a common business subdomain, and through the ratio of intra-module to inter-module dependencies.

Change Impact The scope of change impact — the number of components that must be modified when a requirement changes — is a direct consequence of coupling. Bohner and Arnold (1996) formalized Change Impact Analysis (CIA) as the process of identifying the set of software artifacts that must be modified following a given change, based on the dependency graph of the system. The ripple effect, as described by Arnold and Bohner (1993), characterizes the phenomenon by which an initial change propagates through chains of dependencies, requiring modifications to components with no direct relationship to the original change site. This effect is most pronounced in tightly coupled Monolithic systems and is substantially reduced by enforced module boundaries. In a tightly coupled Monolithic system, a change to grading logic may require modifications to the assignment module, the progress tracking module, and the reporting module simultaneously. In a well-structured Modular Monolith, the same change is confined to the assessment module and its public interface. In a Microservices architecture, the change is isolated to the assessment service, though it may require API contract updates if consuming services depend on the changed behavior. This study operationalizes change impact through the following metric:

$$\text{Change Impact Score} = \frac{\text{Number of components affected}}{\text{Total number of components in the system}}$$

This metric is applied uniformly across all three architectural implementations using four representative change scenarios: the addition of a new data field propagated across system layers; the modification of a business rule within a bounded domain; the introduction of a new end-to-end feature requiring integration across modules; and the restructuring of an existing module boundary. Together, these scenarios provide empirical data on coupling and change impact across architectures, rather than relying solely on theoretical prediction.

### 2.2.5 Scalability Models

Theoretical Foundations Scalability in the context of this study refers to the ability to increase system capacity in response to growing demand. Bondi (2000) defines scalability as the property of a system to handle a grow-

ing amount of work by means of adding resources, and identifies four distinct dimensions: load scalability, space scalability, space-time scalability, and structural scalability. Of these, load scalability — the ability to handle increasing request volumes — and structural scalability — the ability to accommodate the addition of new functional components without architectural reorganization — are the dimensions most directly relevant to the comparative evaluation conducted here. Abbott and Fisher (2015), in The Art of Scalability, provide a complementary model through the Scale Cube, which organizes scaling strategies along three axes. The X-axis represents horizontal duplication: running multiple identical instances of the application behind a load balancer. The Y-axis represents functional decomposition: splitting the application along functional boundaries so that different services can be scaled independently. The Z-axis represents data partitioning: distributing data across instances based on a partition key. This model provides a useful vocabulary for characterizing the scaling capabilities of each architectural style examined in this study. Scaling Characteristics by Architecture Two scalability models are relevant: vertical scaling, which increases the capacity of individual servers, and horizontal scaling, which adds additional instances of services or servers. Monolithic architectures support only application-level horizontal scaling along the X-axis of the Scale Cube: additional instances of the entire application are deployed behind a load balancer. This approach is wasteful when only specific functional areas require additional capacity, as the entire application — including modules with low demand — must be replicated in full. Modular Monolith architectures, as single deployment units, are similarly constrained to X-axis scaling. However, the presence of clearly defined module boundaries preserves structural scalability in the sense of Bondi (2000): the architecture can accommodate the addition of new modules without global reorganization, and the boundary discipline facilitates future extraction of high-demand modules into independent services should operational requirements change. Microservices architectures extend scaling capability to the Y-axis: individual services can be scaled in proportion to their specific demand, without replicating components that are not under load. In the context of an LMS, this means that the assessment service can be independently scaled during high-traffic examination periods without increasing the resource footprint of the course management or content delivery services. This fine-grained scaling efficiency is a primary operational advantage of the Microservices style, though it is realized only at the cost of the infrastructure complexity described in Section 2.3.3.

## 2.3 Comparison Methology

Based on the theoretical principles reviewed in the preceding sections and informed by the existing comparative literature, this project adopts the following five evaluation criteria for the architectural comparison. These criteria are applied consistently across all three change scenarios defined in Chapter 1.

This framework extends the theoretical analysis presented in the comparative literature by grounding each criterion in observable experimental measurements. The change scenarios defined in Chapter 1 — modifying grading logic, adding progress statistics, and preparing for learning analytics integration — are designed to stress-test each criterion across architectures of increasing functional complexity.

### 2.3.1 Degree of Module Separation

Degree of Module Separation refers to the extent to which functional domains within an LMS are encapsulated behind explicit architectural boundaries, preventing unintended coupling and uncontrolled access to shared state. In monolithic architectures, the LMS is packaged and deployed as a single executable unit. Although logical separation (e.g., layered architecture) may exist, modules share the same runtime environment and database, which increases the risk of tight coupling and hidden dependencies.

In contrast, microservices architecture decomposes the LMS into independently deployable services, each aligned with a specific business capability and typically owning its own database schema. This enforces stronger service boundaries and clearer ownership.

In [1], Al-Debagy and Martinek conducted a comparative review of monolithic and microservices architectures and found that microservices significantly enhance modular independence by enforcing service-level isolation and decentralized data management. Similarly, in [2] Dragoni et al. analyzed the evolution of microservices and concluded that service decomposition increases architectural modularity by introducing bounded contexts and independent lifecycle management. Furthermore, in [2], Sastrawan and Suputra performed an empirical comparison and observed stronger inter-component coupling in monolithic systems compared to microservices implementations.

These findings suggest that microservices achieve a higher degree of module separation, particularly beneficial in complex and evolving LMS environments.

### 2.3.2 Scope of Change Impact

Scope of Change Impact measures how many components must be modified when system requirements evolve. In monolithic LMS systems, due to shared codebases and integrated modules, a localized requirement change — such as updating grading logic — may propagate across multiple layers. This increases regression risk and reduces development agility.

In microservices architectures, changes are typically localized within the service responsible for the affected functionality. For example, modifying assessment rules primarily impacts the Assessment Service without requiring changes in unrelated services such as Course or User Management.

In [3], Sastrawan and Suputra conducted experimental comparisons and found that requirement modifications in monolithic systems affected a broader set of components compared to microservices-based systems. Additionally, in [4], Taibi et al. performed a systematic mapping study on microservices patterns and found that service decomposition reduces change propagation by establishing clear service ownership and minimizing cross-service dependencies.

Thus, empirical evidence suggests that microservices reduce the blast radius of change compared to monolithic systems

### 2.3.3 Functional-Level Scalability

Functional-Level Scalability evaluates whether specific LMS functionalities can be scaled independently according to workload demands. In monolithic architectures, scaling typically occurs at the application level, meaning the entire LMS must be replicated even if only one function — such as quiz submission during peak exams — requires additional resources. This can lead to inefficient infrastructure utilization.

Microservices architecture enables fine-grained scalability. Each service can scale independently based on its specific workload characteristics. In [1], Al-Debagy and Martinek analyzed performance characteristics and reported that microservices provide superior scalability flexibility in distributed environments. While monolithic systems may exhibit lower overhead in small-scale deployments, microservices demonstrate greater elasticity under fluctuating loads. Moreover, in [3], Dragoni et al. highlighted that one of the core motivations behind microservices adoption is improved scalability through decentralized and independently scalable services.

Therefore, microservices offer clear advantages in functional-level scalability, particularly in large-scale LMS deployments with uneven workload distribution.

### 2.3.4 Deployment and Operational Complexity

Deployment and Operational Complexity refers to the effort required to build, deploy, monitor, and maintain the system. Monolithic architectures are generally simpler to deploy, as the application is packaged as a single artifact with centralized configuration and monitoring.

However, microservices architectures introduce distributed infrastructure requirements, including service discovery, API gateways, inter-service communication protocols, container orchestration, and distributed monitoring systems.

In [4], Taibi et al. identified increased operational overhead as one of the major challenges of microservices adoption, emphasizing the need for DevOps maturity and automation. Likewise, in [2], Dragoni et al. acknowledged that although microservices enhance modularity and scalability, they significantly increase deployment and operational complexity due to distributed system management.

These findings indicate that while microservices provide architectural benefits, they require more sophisticated infrastructure and operational capabilities compared to monolithic systems.

### 2.3.5 Suitability for System Scale

Suitability for System Scale evaluates how appropriate each architecture is for LMS systems of varying sizes. Monolithic architectures are often well-suited for small to medium-sized LMS deployments due to their simplicity, lower infrastructure cost, and ease of development.

However, as system scale increases — particularly in scenarios involving thousands of concurrent learners, distributed user bases, and high availability requirements — microservices architectures become more advantageous due to their scalability, fault isolation, and independent evolution capabilities. However, the benefits of Microservices must be balanced against increased operational complexity and infrastructure costs. This criterion is assessed qualitatively, informed by experimental results, real-world case studies, and established architectural literature.

In [1], Al-Debagy and Martinek concluded that monolithic systems perform adequately in small-scale environments but face maintainability and scalability limitations as system complexity grows. Additionally, in [4], Taibi et al. emphasized that microservices architectures are more appropriate for large-scale and continuously evolving systems, provided that organizations possess sufficient operational expertise.

Therefore, architectural suitability is context-dependent, and the decision must consider system scale, complexity, and team capabilities.

## 2.4 Relevant Technologies

This section reviews the key technologies adopted across all three architectural implementations in this project. A deliberate decision was made to maintain a uniform technology stack — NestJS, Vue.js, and PostgreSQL — across all three architectures, ensuring that observed differences are attributable to architectural structure rather than technology choice.

### 2.4.1 NestJS Framework

NestJS is a progressive Node.js framework for building server-side applications, built on top of TypeScript and inspired by Angular's modular design principles. Its native support for modules, providers, controllers, and

dependency injection makes it particularly well-suited for implementing all three target architectures within a consistent programming model.

For the Monolithic implementation, NestJS provides a single application context with shared services and a common database connection. For the Microservices implementation, NestJS's built-in microservices transport layer — supporting patterns such as TCP, Redis-based message queues, and gRPC — provides the inter-service communication infrastructure.

The framework's module system is architecturally significant: a NestJS module encapsulates a cohesive set of providers and can explicitly declare its public interface. When inter-module dependencies are forced to go through declared exports, the framework structurally prevents the accidental cross-cutting dependencies that commonly undermine modular designs.

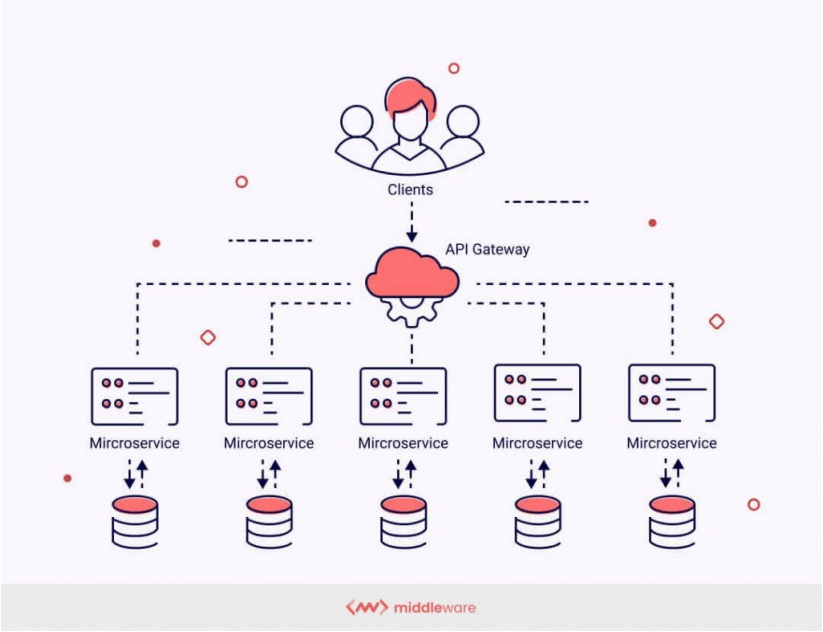



Figure 2.1: Nestjs in Microservice achitecture

### 2.4.2 Vue.js Frontend Framework

Vue.js is a progressive JavaScript framework for building user interfaces, adopted in this project for the frontend layer common to all three architectural variants. Maintaining a single frontend codebase across all three backends allows the study to isolate the impact of backend architectural choices without introducing frontend variability.

In the Monolithic configurations, the Vue.js application communicates with a single backend API endpoint. In the Microservices configuration, an API Gateway aggregates requests from the Vue.js frontend and routes them to the appropriate downstream service. This pattern, referred to as the Backend-for-Frontend (BFF) pattern, is a standard approach in production Microservices deployments.

### 2.4.3 PostgreSQL and Database Strategy

PostgreSQL is used as the primary relational database across all three architectural implementations. However, the database strategy differs meaningfully between architectures and constitutes one of the most substantive structural differences among the three variants.

In the Monolithic architecture, all domain entities — Students, Lecturer, Reviewer, Admin, courses, enrollments, assignments, grades, and progress records — reside in a single shared database schema. This schema is accessed by all application modules, creating a high degree of database-level coupling.

On the other hand, Microservices possesses its special principle: each service owns its own state. The state requirements are resolved through API calls rather than shared database access, providing the highest degree of data isolation but introducing challenges in consistency and cross-service querying.

### 2.4.4 Load Balancing and Reverse Proxies

Load balancing and reverse proxy mechanisms play a relevant but architecturally differentiated role across the three implementations evaluated in this project. A reverse proxy receives incoming client requests and forwards them to one or more backend servers, while also handling concerns such as TLS termination, request routing, and health-based traffic exclusion. In this project, Nginx serves as the reverse proxy across all three architectural variants, providing a consistent ingress layer.

For Monolithic, the reverse proxy configuration is straightforward and identical between the two: Nginx routes all incoming traffic to a single application upstream, applying round-robin load balancing across multiple instances when horizontal scaling is required.

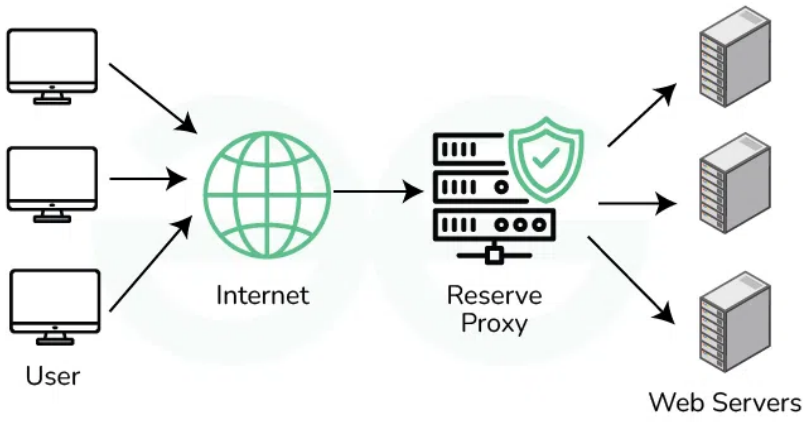



Figure 2.2: Reverse Proxy: Handling Internet Requests for Multiple Servers

In the Microservices architecture, the reverse proxy role expands into two layers. Nginx handles TLS termination and acts as the outermost ingress point, forwarding all requests to a NestJS API Gateway, which in turn applies service-specific routing rules to direct requests to the appropriate downstream service. This two-layer pattern abstracts the internal service decomposition from the Vue.js frontend and enables per-service independent scaling. However, it also introduces additional configuration complexity — a quantifiable dimension in the operational complexity criterion of this study's comparison framework.

### 2.4.5 Docker and Containerization

Docker is used to containerize all architectural implementations, ensuring consistent deployment environments and enabling fair performance comparisons. Each architecture is packaged as a Docker Compose configuration specifying the required services, database instances, and network topology.

For the Monolithic architecture, a single application container is defined alongside a database container. For the Microservices architecture, individual containers are defined per service, along with a shared API Gateway

container and service-specific database containers. This configuration difference itself constitutes a measurable dimension of operational complexity in the comparative evaluation.

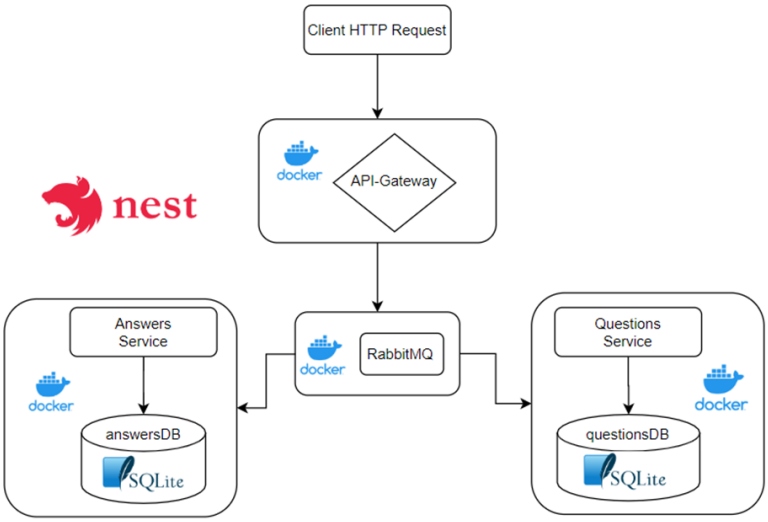



Figure 2.3: Docker: Handling service containerization for microservice (for example)

## 2.5 Summary

2.5 Summary This chapter has reviewed the relevant literature on software architecture in LMS contexts, established the theoretical foundations of Monolithic, Modular Monolith, and Microservices architectural styles, and defined the comparison framework applied in the experimental phase of this project. The key insight from the literature is that architectural choices involve trade-offs along multiple dimensions — maintainability, operational complexity, scalability, and change impact — and that the optimal choice is context-dependent rather than universal. The technologies adopted in this project — NestJS, Vue.js, PostgreSQL, and Docker — provide a stable, uniform experimental platform that isolates architectural differences as the primary independent variable. The five-criterion comparison framework operationalizes theoretical concepts of coupling, cohesion, and scalability into measurable experimental dimensions. The subsequent chapters translate these theoretical foundations and the defined comparison framework into a concrete system design (Chapter 3), three architectural implementations (Chapter 4), and a systematic experimental evaluation (Chapter 5), yielding empirical evidence on the practical trade-offs of each architectural style in an LMS context. Etiam ac leo a risus tristique nonummy. Donec dignissim tincidunt nulla. Vestibulum rhoncus molestie odio. Sed lobortis, justo et pretium lobortis, mauris turpis condimentum augue, nec ultricies nibh arcu pretium enim. Nunc purus neque, placerat id, imperdiet sed, pellentesque nec, nisl. Vestibulum imperdiet neque non sem accumsan laoreet. In hac habitasse platea dictumst. Etiam condimentum facilisis libero. Suspendisse in elit quis nisl aliquam dapibus. Pellentesque auctor sapien. Sed egestas sapien nec lectus. Pellentesque vel dui vel neque bibendum viverra. Aliquam porttitor nisl nec pede. Proin mattis libero vel turpis. Donec rutrum mauris et libero. Proin euismod porta felis. Nam lobortis, metus quis elementum commodo, nunc lectus elementum mauris, eget vulputate ligula tellus eu neque. Vivamus eu dolor. ======== This chapter has reviewed existing real-time messaging solutions, relevant technologies, and theoretical principles that underpin the design of scalable messaging systems. The insights gained from this literature review will inform the design and implementation choices made

in subsequent chapters.

# Chapter 3

# Requirements Analysis

This chapter translates business needs into precise, actionable software requirements. It follows a structured approach: identifying users and stakeholders, specifying functional and non-functional requirements, modeling the domain, and addressing constraints, risks, and traceability. The goal is to ensure the system meets both explicit and implicit expectations for a scalable, real-time messaging platform.

## 3.1 Stakeholders and User Personas

The identification and analysis of stakeholders is a foundational step in requirements engineering, as highlighted in several studies on large-scale messaging systems [**?**, **?**, **?**].

The system targets organizations and platforms that require low-latency, large-scale messaging, such as customer support platforms, trading dashboards, collaborative editing tools, and social chat applications. The following primary stakeholder groups are identified:

- The following list outlines the main categories of stakeholders who interact with or are impacted by the system. Each group has distinct needs and expectations, which must be addressed to ensure the platform's success and adoption.
- **End Users (Mobile/Web Clients):** Individuals who send/receive messages in private or group channels with real-time delivery and read receipts.
- **Tenant Administrators:** Configure workspaces, manage users/roles, enforce security, and monitor usage.
- **System Operators (SRE/DevOps):** Maintain uptime, capacity planning, scaling, and incident response.
- **Product Owners (Business Stakeholders):** Define business objectives, SLAs, compliance, and roadmap.
- **Third-party Integrators:** Developers or vendors who extend the system via APIs, plugins, or bots.
- **Compliance Officers:** Ensure the system meets regulatory, privacy, and audit requirements.

### Representative Personas

The following personas are designed to represent typical users and stakeholders of the system. By understanding their goals and challenges, we can better align the system's features and priorities with real-world needs.

The use of personas is a widely adopted practice to ensure that requirements reflect real-world user needs, as discussed in [**?**, **?**, **?**].

**Persona A — "Alice, the Support Agent"**: Alice simultaneously handles multiple customer conversations. She needs instantaneous message updates, typing indicators, and delivery confirmations to provide responsive support.

**Persona B — "Ben, the Admin"**: Ben manages a 5,000-user workspace. He needs robust role-based access control, audit logs, and rate limits to protect the system from abuse and ensure compliance.

**Persona C — "Cara, the SRE"**: Cara must observe connection counts, message throughput, node health, and error rates; she requires autoscaling hooks and circuit breaking to maintain high availability.

**Persona D — "Dan, the Integrator"**: Dan builds a chatbot that listens to messages and posts automated replies. He needs a stable API, event hooks, and clear documentation.

**Persona E — "Eva, the Compliance Officer"**: Eva audits message retention and access logs to ensure GDPR and data residency compliance.

Personas anchor requirements in realistic workflows and help justify non-functional needs such as latency, throughput, resilience, and compliance.

## 3.2 Use Case Scenarios

To illustrate how the system will be used in practice, we present several representative scenarios. These use cases demonstrate the breadth of interactions and requirements that the platform must support. The following use cases are informed by best practices in real-time system design and are supported by recent research and industry documentation [**?**, **?**, **?**, **?**]. To clarify requirements, we describe representative user scenarios:

- **UC-1: Real-Time Customer Support**
  Alice receives a new support ticket. She joins a private channel with the customer, exchanges messages in real time, and shares a file attachment. The customer sees typing indicators and read receipts. The conversation is archived for future reference.

- **UC-2: Admin Onboarding**
  Ben creates a new channel for a project team, sets access permissions, invites members, and configures message retention policies. He reviews audit logs to verify no unauthorized access.

- **UC-3: Outage Recovery**
  Cara detects a spike in error rates. She uses the metrics dashboard to identify overloaded nodes, triggers autoscaling, and verifies that clients reconnect with minimal disruption.

- **UC-4: Bot Integration**
  Dan registers a webhook to receive message events. His bot posts automated responses and logs interactions for analytics.

- **UC-5: Compliance Audit**
  Eva exports access logs and verifies that message retention and deletion policies are enforced for a specific tenant.

## 3.3 Functional Requirements

Functional requirements define the specific behaviors and capabilities that the system must provide. These requirements are derived from user needs, business objectives, and technical constraints, and are organized by domain for clarity.

This section specifies the essential capabilities of the messaging system. Requirements are grouped by domain and expressed as user stories with acceptance criteria. Where appropriate, edge cases and error handling are included.

### 3.3.1 Core Messaging

At the heart of the platform is the ability to exchange messages in real time. The following requirements are grounded in established messaging protocols and architectures [**?**, **?**, **?**, **?**], ensuring that users can communicate efficiently, reliably, and securely, with support for essential messaging features and robust error handling.

- **FR-1 Send Message:** As an *End User*, I can send a text message to a channel or direct conversation, so that recipients receive it in real time.
  - *Acceptance:* Message is delivered via WebSocket within the latency target (see §3.6.1); server persists the message with a unique ID; the sender immediately sees the message in the timeline with a local timestamp.
- **FR-2 Receive Message:** As an *End User*, I receive inbound messages instantly without manual refresh.
  - *Acceptance:* When another user sends a message to my active channel, my client appends it to the timeline and triggers visual notification.
- **FR-3 Read Receipts:** As an *End User*, I can see per-message delivery/read status.
  - *Acceptance:* Server maintains per-user read offsets; UI shows delivered/seen indicators; updates propagate in real time.
- **FR-4 Typing Indicators:** As an *End User*, I can observe when others are typing in the same channel.
  - *Acceptance:* Client emits throttled "typing" events; server fans out to channel members; indicators auto-expire after inactivity.
- **FR-4.1 File Attachments:** As an *End User*, I can send and receive file attachments (images, documents) in messages.
  - *Acceptance:* Files are uploaded securely, scanned for viruses, and linked in the message timeline; download links expire after a configurable period.
- **FR-4.2 Message Editing/Deletion:** As an *End User*, I can edit or delete my own messages within a configurable time window.
  - *Acceptance:* Edits and deletions are reflected in real time; audit logs retain original content for compliance.
- **FR-4.3 Error Handling:** If message delivery fails (e.g., network loss), the client retries with exponential backoff and displays a clear error state.
  - *Acceptance:* Users are notified of undelivered messages and can manually retry.

### 3.3.2 Channels, Membership, and Presence

Channels and membership management are fundamental to organizing conversations and controlling access. Presence information enhances the user experience by indicating availability and activity. These requirements are consistent with recommendations from recent studies on scalable messaging and presence systems [**?**,**?**,**?**,**?**].

- **FR-5 Channel Management:** Admins can create public/private channels and manage membership.
- **FR-6 Presence:** The system reflects online/offline/away states and last-seen timestamps per user.
- **FR-7 Invitations:** Users can invite others to channels they own or administer.
- **FR-7.1 Channel Roles:** Each channel supports roles (owner, admin, member, guest) with configurable permissions.
- **FR-7.2 Channel Archiving:** Admins can archive channels, making them read-only and hidden from active lists.
- **FR-7.3 Presence Subscriptions:** Clients can subscribe/unsubscribe to presence updates for specific users or channels to optimize bandwidth.

### 3.3.3 Search and History

Efficient access to historical messages and the ability to search through conversations are critical for productivity and compliance. These features are emphasized in both academic and industry sources [**?**, **?**, **?**].

- **FR-8 Message History:** Users can scroll back to view historical messages with efficient pagination.
- **FR-9 Search:** Users can search messages by keyword, sender, channel, and time range.
- **FR-9.1 Export History:** Users with permission can export message history for a channel or conversation in a standard format (e.g., CSV, JSON).

### 3.3.4 Security and Access Control

Security is paramount in any messaging system, especially one designed for organizational use. The requirements in this section are informed by best practices and standards for secure messaging platforms [**?**, **?**, **?**, **?**], focusing on authentication, authorization, auditability, and data retention, providing a foundation for trust and compliance.

- **FR-10 Authentication:** The system supports token-based authentication (e.g., JWT) and WebSocket authentication during the handshake.
- **FR-11 Authorization:** Role-Based Access Control (RBAC) permits fine-grained channel access and admin features.
- **FR-12 Audit Trail:** Administrative actions and critical security events are logged and retrievable by authorized staff.
- **FR-12.1 Session Management:** Users can view and revoke active sessions/devices from their account settings.
- **FR-12.2 Data Retention:** Message and log retention policies are configurable per tenant and enforced automatically.

### 3.3.5 Administration and Observability

To maintain high availability and performance, the system must offer comprehensive administration and observability features. These requirements are supported by research on operational visibility and system monitoring in distributed environments [**?**, **?**, **?**].

- **FR-13 Metrics:** Operators can view metrics (active connections, messages/sec, latency percentiles, error rates).
- **FR-14 Rate Limiting:** The system enforces per-user and per-IP rate limits on message send and connection attempts.
- **FR-15 Configuration:** Admins can configure workspace limits (max channel members, attachment sizes).
- **FR-16 Alerting:** Operators receive alerts for SLA violations, abnormal error rates, or resource exhaustion.
- **FR-17 Maintenance Mode:** The system supports a maintenance mode with user notifications and graceful connection draining.

## 3.4 Regulatory and Compliance Requirements

Modern messaging systems must adhere to a variety of regulatory and compliance standards. This section summarizes the key obligations and features required to meet legal and organizational policies. Compliance with regulatory standards is a recurring theme in the literature on enterprise messaging systems [**?**, **?**, **?**]. The system must comply with relevant regulations and standards, including:

- **GDPR:** Support data subject requests (export, deletion), minimize PII, and provide data residency options.
- **Auditability:** Maintain tamper-evident logs of administrative and security events.
- **Accessibility:** Meet WCAG 2.1 AA guidelines for user interfaces.
- **Localization:** Support multiple languages and regional formats for dates, times, and numbers.

## 3.5 Internationalization and Accessibility

To serve a global and diverse user base, the system must be designed with internationalization and accessibility in mind. The following requirements ensure that the platform is usable and inclusive for all users, regardless of language or ability. Internationalization and accessibility are increasingly important in modern software systems, as highlighted in recent guidelines and technical documentation [**?**, **?**, **?**].

- **I18N-1:** All user-facing text is externalized for translation; right-to-left (RTL) languages are supported.
- **I18N-2:** Date/time formats and time zones are user-configurable.
- **A11Y-1:** The UI is navigable via keyboard and screen readers; color contrast meets accessibility standards.
- **A11Y-2:** Real-time notifications are announced to assistive technologies.

### 3.5.1 Representative Use Case Diagram

Figure 3.1 presents a high-level use case model, summarizing relationships among actors and core features.

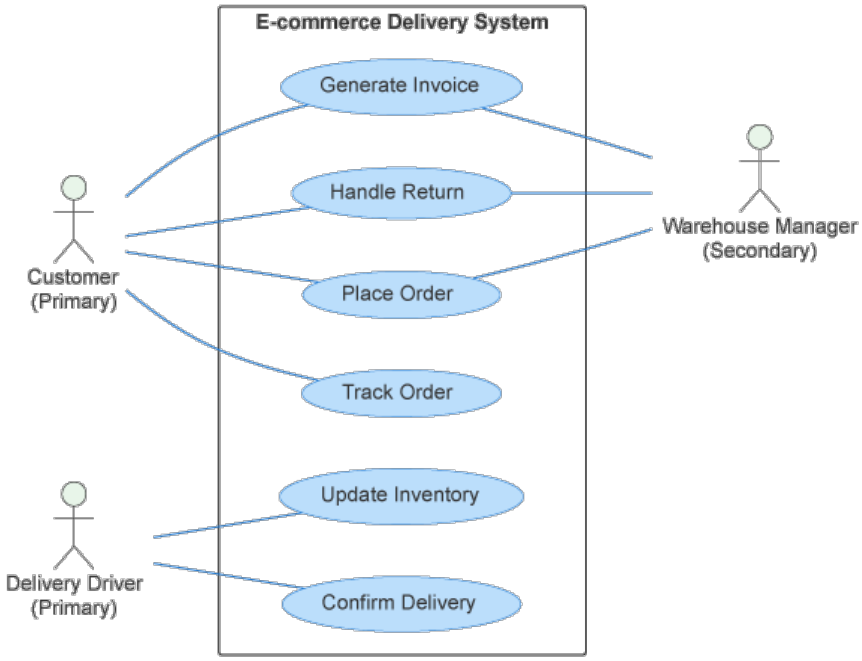



Figure 3.1: High-level use case diagram for real-time messaging.

## 3.6 Non-Functional Requirements

Non-functional requirements (NFRs) define the quality attributes of the system, such as performance, reliability, security, and extensibility. These requirements are critical for ensuring that the platform not only works, but works well under real-world conditions.

### 3.6.1 Performance and Scalability

The system must deliver messages with minimal delay and scale to support large numbers of concurrent users. These performance and scalability targets are based on established benchmarks and architectural patterns [**?**, **?**, **?**].

- **NFR-1 Latency:** 95% of message deliveries should complete end-to-end within $< 150$ ms on a regional deployment; 99% within $< 300$ ms.
- **NFR-2 Throughput:** The system should handle at least 50,000 concurrent WebSocket connections per region and sustain $\geq 10{,}000$ messages/sec with linear horizontal scaling.
- **NFR-3 Fan-out Efficiency:** Broadcasting to a channel of 1,000 members should not exceed $2\times$ the median single-recipient latency.

### 3.6.2 Availability and Reliability

High availability and reliability are essential for a real-time messaging platform. These requirements are informed by studies on distributed system resilience and cloud-native patterns [**?**, **?**, **?**].

- **NFR-4 Availability:** Target service availability of 99.9% monthly, with zero single points of failure in the real-time path.
- **NFR-5 Resilience:** Node failures or AZ outages trigger automatic failover; clients auto-reconnect with exponential backoff and session resumption.
- **NFR-6 Durability:** Persisted messages must be stored with replication factor $\geq 3$ or equivalent durability guarantees.

### 3.6.3 Security and Privacy

Protecting user data and ensuring secure communication are core principles of the system. These non-functional requirements are supported by both academic and industry sources [**?**, **?**, **?**].

- **NFR-7 Transport Security:** WebSocket over TLS (wss://) is mandatory; tokens never transmitted in query strings.
- **NFR-8 Access Control:** All administrative operations require privileged roles; sensitive actions logged with actor, timestamp, IP, and request ID.
- **NFR-9 Data Protection:** Personally identifiable information (PII) is minimized; retention policies are configurable per tenant.

### 3.6.4 Operability and Observability

Effective monitoring and troubleshooting are vital for maintaining system health. The requirements below are based on best practices for observability in real-time and distributed systems [**?**, **?**, **?**].

- **NFR-10 Metrics:** Export Prometheus-format metrics for connections, publish latency (P50, P95, P99), dropped frames, and reconnect rates.
- **NFR-11 Tracing:** Distributed tracing across gateway $\rightarrow$ broker $\rightarrow$ persistence services to localize latency hotspots.
- **NFR-12 Logging:** Structured JSON logs with correlation IDs; log sampling prevents I/O bottlenecks at peak traffic.

### 3.6.5 Compatibility and Extensibility

The platform should be compatible with a wide range of clients and support future growth through extensibility. These goals are consistent with recommendations from recent technical documentation and case studies [**?**, **?**, **?**].

- **NFR-13 Client Compatibility:** Support modern browsers and mobile runtimes with WebSocket; provide fallback guidance (no long polling by default).
- **NFR-14 Extensibility:** Plugin hooks for moderation bots, keyword alerts, and custom retention policies.

## 3.7 Domain Model and Data Requirements

The domain model defines the core entities and relationships that underpin the system's functionality. The approach taken here is informed by established practices in data modeling for messaging platforms [**?**, **?**, **?**].

We define the essential entities and their relationships. The following table summarizes key attributes for each entity:

Table 3.1: Key Entity Attributes

| Entity | Attribute | Description |
|---|---|---|
| User | user_id | Unique identifier |
| | profile | Display name, avatar, contact info |
| | presence | Online/offline/away state |
| | roles | Global and per-channel roles |
| Channel | channel_id | Unique identifier |
| | name | Channel display name |
| | visibility | Public/private |
| | settings | Retention, max members, etc. |
| Membership | user_id, channel_id | Composite key |
| | role | Owner/admin/member/guest |
| | join_time | Timestamp of joining |
| Message | message_id | Unique identifier |
| | sender | user_id reference |
| | channel | channel_id reference |
| | content | Text, attachment link, metadata |
| | timestamps | Sent, delivered, read times |
| | status | Delivered/read/edited/deleted |

**Key Entities** The following key entities form the backbone of the system's data model. Each entity is described by its primary attributes and its role within the platform.

- **User:** *user_id*, profile, presence state, roles.
- **Channel:** *channel_id*, name, visibility (public/private), settings.
- **Membership:** link between *user* and *channel*, including role and join time.
- **Message:** *message_id*, sender, channel, content, timestamps, delivery/read states.

## 3.8 API Requirements (High Level)

The system exposes both WebSocket events and RESTful endpoints to facilitate client-server communication. This section only lists the main API groups; detailed endpoints and events will be presented in Chapter 4 (System Design).

- **WebSocket Events:** Events serving real-time communication between client and server (sending/receiving messages, status updates, typing, etc.).
- **REST Endpoints:** Control APIs for channel management, user management, configuration, history retrieval, statistics, etc.

Details of the structure, event flows, and endpoints will be presented in the system design chapter.

## 3.9 Future Requirements and Planned Extensions

To remain competitive and adaptable, the system is designed with future enhancements in mind. The following potential extensions illustrate directions for ongoing development and innovation. To ensure long-term viability, the system is designed for extensibility. Potential future requirements include:

- **Federation:** Support for cross-organization messaging and federation with other platforms.
- **Voice/Video Integration:** Real-time audio/video channels alongside text.
- **Advanced Moderation:** AI-powered content moderation and abuse detection.
- **Custom Bots/Plugins:** Marketplace for third-party extensions.
- **Offline Messaging:** Store-and-forward for intermittently connected clients.

## 3.10 Constraints and Assumptions

Every system operates within certain constraints and is built on specific assumptions. This section documents those factors, which may impact design decisions and system behavior.

- **Assumptions:** Clients maintain stable network connectivity; clocks are approximately synchronized for display ordering; TLS termination occurs at the edge proxy.
- **Constraints:** Browser sandbox restrictions; per-socket memory limits on fan-out buffers; regulatory constraints on data retention for certain tenants.

## 3.11 Risk Analysis (Selected)

Identifying and mitigating risks is crucial for the success of any large-scale system. The following risks have been identified as particularly relevant to this project, along with proposed mitigation strategies.

- **R-1 Hot Channel Fan-out:** A channel with tens of thousands of subscribers can cause uneven load. *Mitigation:* shard-by-channel, layered fan-out, and adaptive batching.
- **R-2 Thundering Reconnect:** Regional outage triggers mass reconnects. *Mitigation:* jittered exponential backoff, connection fencing, and token TTL staggering.
- **R-3 State Coordination:** Presence and read receipts across nodes can diverge. *Mitigation:* central pub/sub (e.g., Redis) with idempotent updates and conflict resolution rules.

## 3.12 Requirements Traceability Matrix (Excerpt)

Traceability ensures that all requirements are linked to business objectives and can be verified through testing or analysis. The following matrix provides a sample mapping for key requirements.

Table 3.2: Traceability matrix (excerpt)

| Business Objective | Requirement IDs | Verification |
|---|---|---|
| Low-latency messaging | FR-1, FR-2; NFR-1 | Load tests measure P95 $<$ 150 ms; integration tests validate real-time delivery. |
| Operational visibility | FR-13, FR-15; NFR-10–12 | Metrics endpoint audited; dashboards show connection counts and latency percentiles. |
| Secure access | FR-10–12; NFR-7–9 | Security tests for TLS, RBAC, audit logs; pen-test checklist passed. |

## 3.13 Summary

This chapter formalized who the system serves, what it must do, and how it must behave under load and failure. The summary and forward reference are consistent with best practices in requirements documentation [**?**, **?**, **?**]. The next chapter (see Chapter 4) will translate these requirements into a concrete system architecture, data model, and API/service design.

# Chapter 4

# System Design

## 4.1 Overview of the System Architecture

### 4.1.1 Architectural Goals

The primary goals of the system architecture are to achieve low-latency, high-throughput real-time messaging at scale, ensure reliability and security, and support extensibility for future features. The architecture is designed to be horizontally scalable, resilient to failures, and maintainable for rapid development cycles.

### 4.1.2 High-Level Architecture Diagram

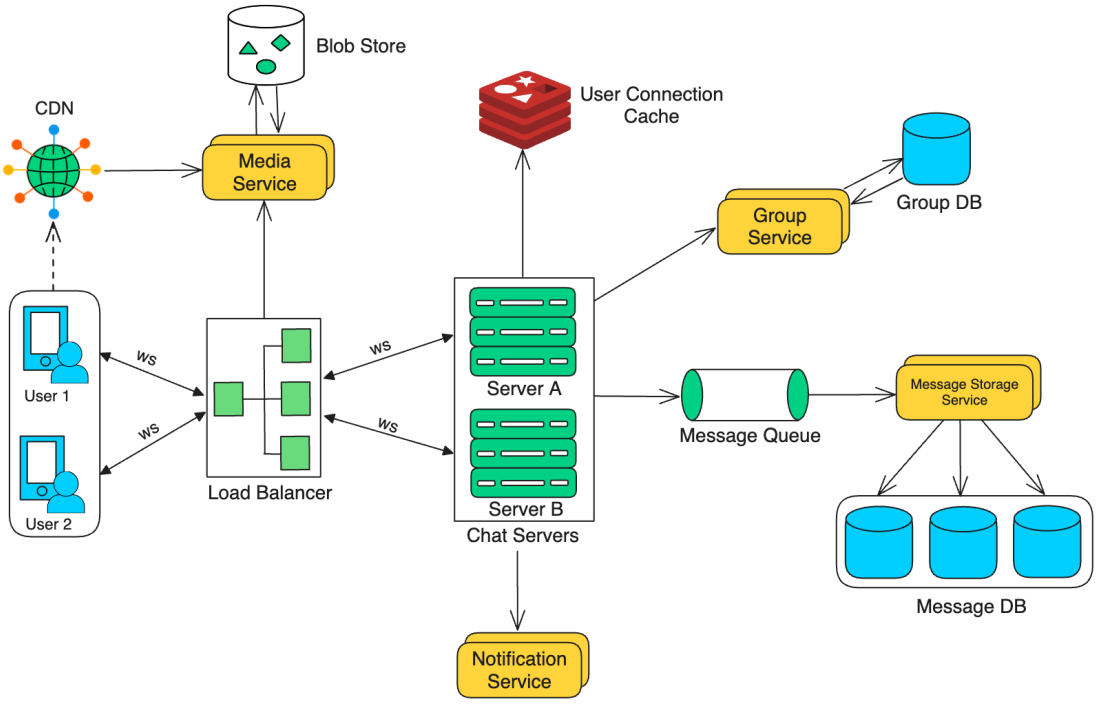



Figure 4.1: High-level system architecture for scalable real-time messaging.

The provided High-Level Architecture Diagram illustrates a robust, scalable, and distributed Real-time Communication (RTC) System. The infrastructure is designed to handle high concurrency and large volumes of data through a decoupled microservices-based approach.

**1. Real-time Connectivity and Session Management**

The system utilizes WebSockets (WS) to facilitate low-latency, bi-directional communication between clients and the backend. A Load Balancer serves as the entry point, distributing incoming traffic across multiple instances of Chat Servers (e.g., Server A and Server B) to ensure high availability. To manage state in a distributed environment, a User Connection Cache (typically implemented using an in-memory store like Redis) tracks active sessions and maps specific users to their respective host servers, enabling efficient message routing.

**2. Message Persistence and Asynchronous Processing**

To decouple real-time delivery from long-term storage, the architecture incorporates an asynchronous pipeline: Message Queue acts as a buffer to ingest high-frequency message streams, ensuring system reliability during peak traffic and preventing the storage layer from becoming a bottleneck. The Message Storage Service consumes data from the queue and manages the persistence logic. The Message DB is a distributed database cluster (likely employing database sharding) designed to store vast quantities of historical chat data while maintaining performant read/write operations. A Group Service manages group metadata and memberships, backed by its own specialized database.

**3. Media Handling and Content Delivery**

The architecture separates signaling (messages) from heavy media assets. Media Service and Blob Store handle file uploads (e.g., images, videos), storing them in a Blob Store (such as AWS S3 or Google Cloud Storage). A Content Delivery Network (CDN) caches and serves media assets from edge locations closer to end-users, minimizing latency and reducing load on origin servers.

**4. Push Notifications**

A Notification Service is integrated to handle asynchronous alerts, ensuring users remain engaged by delivering push notifications for incoming messages when they are offline or when the application is in the background.

**Key Architectural Strengths:** Scalability is achieved through horizontal scaling at every layer. Resilience is ensured by using message queues and distributed caching to prevent single points of failure. Performance is optimized by offloading media to a CDN and using WebSockets for signaling, resulting in a responsive user experience.

### 4.1.3 Data Flow Overview

The system processes user actions (e.g., sending messages, joining channels) through a series of layers: the WebSocket gateway receives client events, which are validated and routed to backend services. Messages are persisted, broadcast via a pub-sub broker, and delivered to all relevant clients. Control plane operations (e.g., channel management) use REST APIs.

### 4.1.4 Main Sequence Diagrams

To clarify the main flows in the system, the following sequence diagrams illustrate the interactions for key scenarios. (You may use PlantUML or external tools to generate the images.)

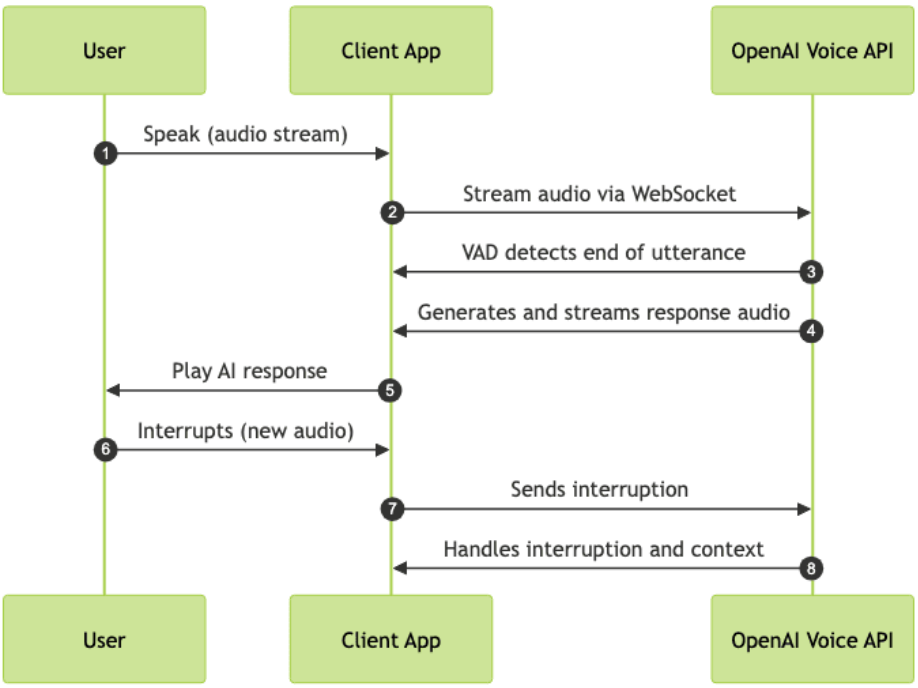



Figure 4.2: Sequence diagram: User sends a message

### a) Send Message Flow

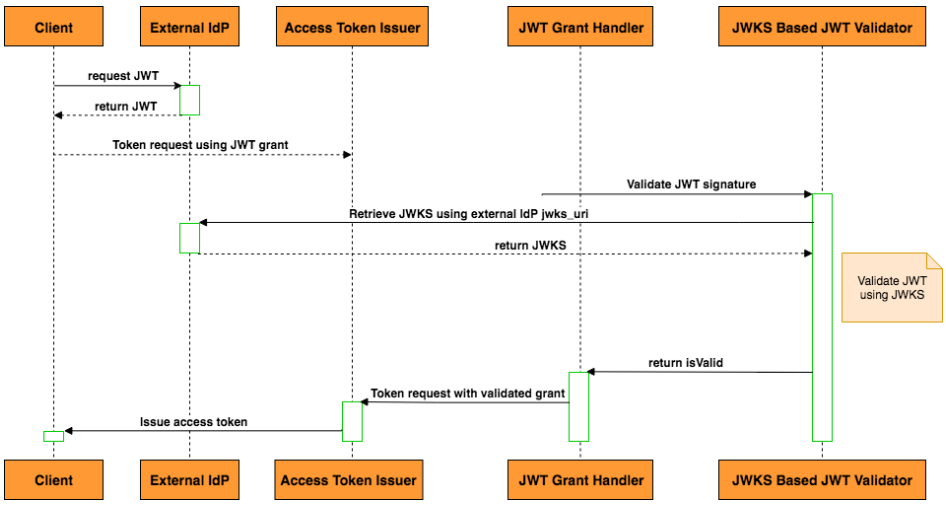



Figure 4.3: Sequence diagram: WebSocket authentication

### b) WebSocket Authentication Flow

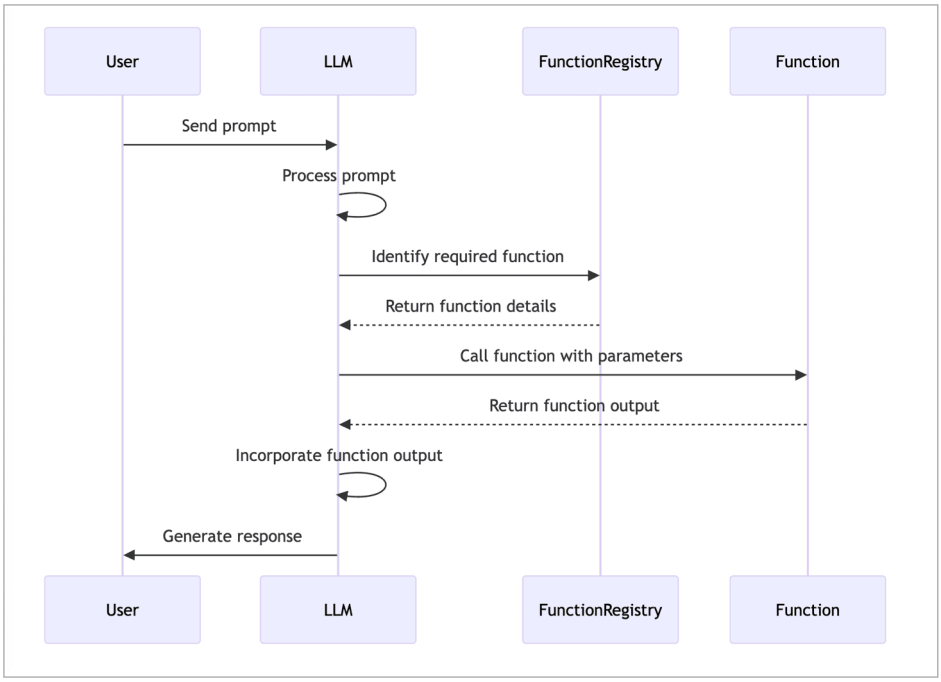



Figure 4.4: Sequence diagram: Presence update

**c) Presence Update Flow**

### 4.1.5 Key Architectural Decisions (KADs)

- **Event-driven, non-blocking Node.js backend** for high concurrency.
- **WebSocket protocol** for real-time, bidirectional communication.
- **Horizontal scaling** via stateless services and distributed brokers.
- **Separation of real-time and control plane** for performance and security.
- **Centralized authentication and RBAC** for secure access control.
- **Observability and metrics** built-in for operational excellence.

## 4.2 System Components and Responsibilities

The system is decomposed into modular components, each with clear responsibilities:

### 4.2.1 WebSocket Gateway Layer

Handles client WebSocket connections, authentication, session management, and initial event routing. Ensures secure, persistent, and low-latency connections.

### 4.2.2 Real-Time Message Service

Processes inbound and outbound messages, applies business logic (e.g., filtering, validation), and interacts with the pub-sub broker for message fan-out.

### 4.2.3 Channel and Membership Service

Manages channels, memberships, roles, and permissions. Provides APIs for channel creation, joining, and membership updates.

### 4.2.4 Persistence Layer (Database + Storage)

Stores user, channel, membership, and message data. Ensures durability, consistency, and efficient querying. May use a combination of relational and NoSQL databases, plus object storage for attachments.

### 4.2.5 Message Broker / Pub-Sub Layer

Distributes messages to all relevant nodes and services. Enables horizontal scaling and decouples message producers from consumers. Typically implemented with Redis, NATS, or Kafka.

### 4.2.6 Authentication & Authorization Service

Manages user authentication (e.g., JWT), token issuance, validation, and role-based access control (RBAC) enforcement.

### 4.2.7 Admin & Observability Layer

Provides dashboards, metrics, logs, and alerting for operators and administrators. Supports monitoring, troubleshooting, and auditing.

## 4.3 Detailed Backend Architecture

### 4.3.1 Node.js Service Architecture (Event-Driven Model)

The backend leverages Node.js's event-driven, non-blocking I/O model to handle thousands of concurrent connections efficiently. Each service is stateless and communicates via APIs or message queues.

### 4.3.2 Horizontal Scaling Strategy (Clustering, Multi-node)

Multiple Node.js instances are deployed across nodes. Clustering and container orchestration (e.g., Kubernetes) enable dynamic scaling. Shared state is minimized; distributed caches and brokers synchronize transient data.

### 4.3.3 Load Balancing and Traffic Routing

Traffic is distributed using a reverse proxy (e.g., Nginx, HAProxy) with support for sticky sessions (if session affinity is needed) or stateless routing. Load balancers monitor node health and route traffic accordingly.

#### Sticky Sessions vs Stateless Routing

Sticky sessions bind a client to a specific gateway for session persistence, while stateless routing allows any gateway to serve any client. Stateless routing is preferred for scalability, but sticky sessions may be used for certain features.

**Reverse Proxy Setup (Nginx/HAProxy)**

Reverse proxies terminate TLS, balance traffic, and provide DDoS protection. Configuration includes health checks, rate limiting, and WebSocket upgrade support.

### 4.3.4 Connection Lifecycle Management

- **WebSocket handshake:** Secure upgrade from HTTP(S) to WebSocket, with authentication.
- **Reconnection logic:** Clients auto-reconnect with exponential backoff after disconnects.
- **Heartbeat/Keep-alive:** Periodic ping/pong frames detect dead connections and maintain session state.

## 4.4 Data Design

### 4.4.1 Database Model Overview

The data model is designed for efficient storage and retrieval of messages, users, channels, and memberships. Normalization is balanced with denormalization for performance.

### 4.4.2 Database Analysis and Design

**Database Technology Selection**

The system requires a database solution that supports high write throughput, low-latency reads, horizontal scalability, and strong consistency for core messaging data. After evaluating several options, the following choices were made:

- **Message Store:** A distributed NoSQL database (e.g., MongoDB, Cassandra, or Amazon DynamoDB) is selected for storing chat messages due to its ability to handle large volumes of semi-structured data, automatic sharding, and high availability. These databases support horizontal scaling and flexible schema evolution, which are essential for real-time messaging workloads.
- **User, Channel, and Membership Data:** A relational database (e.g., PostgreSQL or MySQL) is used for structured data with strong consistency requirements, such as user profiles, channel metadata, and membership relations. This enables complex queries, referential integrity, and transactional updates.
- **Caching and Pub/Sub:** Redis is employed for caching hot data (presence, recent messages) and for implementing the pub-sub pattern to distribute real-time events across nodes.
- **Media Storage:** Large files (images, videos) are stored in a cloud object storage service (e.g., AWS S3), decoupled from the main database to optimize performance and cost.

**Design Rationale**

- **NoSQL for Messages:** Message data is append-only, high-volume, and does not require complex joins, making NoSQL a natural fit. Sharding by channel or tenant enables linear scaling.
- **Relational for Metadata:** User and channel data benefit from ACID transactions and relational integrity, supporting features like permissions, roles, and analytics.

- **Hybrid Approach:** Combining NoSQL and SQL leverages the strengths of both paradigms, ensuring performance and data integrity where needed.
- **Redis for Real-Time:** Redis provides sub-millisecond latency for presence and pub-sub, critical for real-time UX.

#### Schema Design and Key Structures

The following table summarizes the main entities, their storage technology, and key fields:

Table 4.1: Core Data Entities and Storage Technologies

| Entity | Storage | Partition Key | Key Fields |
|---|---|---|---|
| User | PostgreSQL | user_id | username, email, password_hash, profile, roles, status |
| Channel | PostgreSQL | channel_id | name, type, settings, created_at |
| Membership | PostgreSQL | (user_id, channel_id) | role, join_time, permissions |
| Message | MongoDB / DynamoDB | channel_id | message_id, sender_id, content, timestamps, attachments, reactions |
| Delivery / Read State | MongoDB / DynamoDB | message_id | user_id, delivered, read_at |
| Presence | Redis | user_id | online_status, last_seen |
| Media | Amazon S3 | file_id | url, owner, type, size, uploaded_at |

#### Sharding and Partitioning Strategy

To ensure scalability, the message store is sharded by channel_id (or tenant in multi-tenant deployments). This distributes write and read load evenly and allows for independent scaling of hot channels. User and channel tables in the relational database are indexed on primary keys and frequently queried fields. Redis keys are namespaced by user or channel for efficient lookup.

#### Replication and High Availability

All databases are deployed in multi-AZ (Availability Zone) configurations. NoSQL clusters use replica sets or multi-region replication for durability. SQL databases employ streaming replication and automated failover. Redis is configured with sentinel or cluster mode for resilience.

#### Data Consistency and Integrity

- **Messages:** Eventual consistency is acceptable for message delivery, but strong consistency is enforced for message creation and deletion.
- **User/Channel:** Strong consistency is required for authentication, permissions, and membership changes.
- **Media:** Metadata is stored in the database, while the actual file is stored in object storage with signed URLs for secure access.

#### Comparison of Database Alternatives

The following table compares candidate database technologies for the message store:

Table 4.2: Comparison of NoSQL Databases for Message Storage

| Database | Scalability | Consistency | Operational Complexity | Best Use Case |
|---|---|---|---|---|
| MongoDB | Horizontal (sharding), flexible schema | Tunable (eventual/strong) | Moderate (requires ops expertise) | General-purpose, flexible queries |
| Cassandra | Linear, multi-region | Eventual | High (complex ops, tuning) | High write throughput, global scale |
| DynamoDB | Fully managed, auto-sharding | Tunable (eventual/strong) | Low (managed service) | Serverless, predictable workloads |

**Summary of Database Design Decisions**

The hybrid approach leverages the strengths of each technology: NoSQL for scalable message storage, SQL for structured metadata, Redis for real-time features, and object storage for media. This design ensures the system can meet demanding requirements for scalability, reliability, and user experience in a real-time messaging context.

### 4.4.3 Entity–Relationship Diagram (ERD)

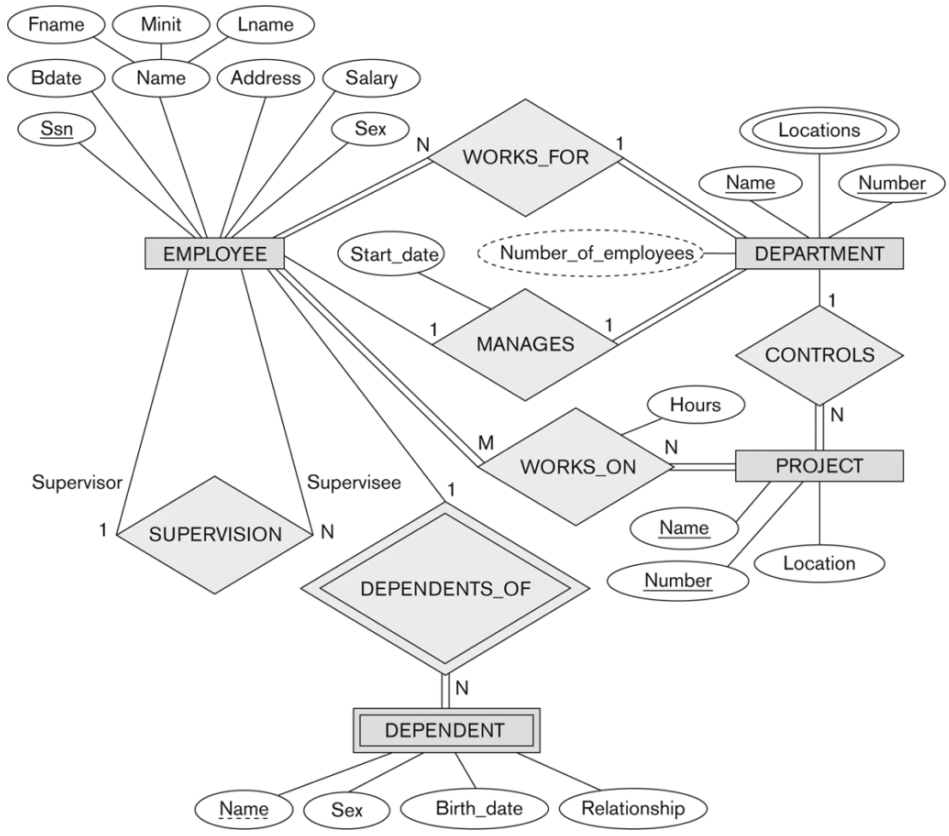



Figure 4.5: Entity–Relationship Diagram for core data entities.

### 4.4.4 Key Entities and Schemas

- **User:** user_id, profile, credentials, presence, roles

- **Channel:** channel_id, name, type, settings
- **Membership:** user_id, channel_id, role, join_time
- **Message:** message_id, sender, channel, content, timestamps, delivery/read state
- **Delivery/Read State:** message_id, user_id, delivered, read_at

### 4.4.5 Data Partitioning & Indexing Strategy

Data is partitioned by tenant or channel for scalability. Indexes are created on frequently queried fields (e.g., channel_id, user_id, timestamps) to optimize performance.

### 4.4.6 Caching Strategy (Redis)

Redis is used for caching hot data (e.g., presence, recent messages) and for pub-sub event distribution. TTLs and eviction policies are tuned for workload patterns.

## 4.5 API & Protocol Design

This section describes the design of the APIs and communication protocols that enable real-time messaging, control operations, and integration with external systems. The API and protocol design is based on recommendations from [**?**, **?**, **?**, **?**] to ensure robust and flexible client-server interactions.

### 4.5.1 WebSocket Protocol Design

The WebSocket protocol is central to delivering low-latency, bidirectional communication between clients and the backend. The following conventions and structures are adopted to standardize event handling and error reporting.

- **Event Naming Convention:** Consistent, namespaced event names (e.g., msg.send, presence.update).
- **Message Frame Structure:** JSON objects with type, payload, and metadata fields.
- **Error Frame Structure:** Standardized error codes and messages for client handling.

### 4.5.2 WebSocket Event Flows

The system defines clear event flows for all real-time interactions, ensuring reliable message delivery, presence updates, and user experience features. Key flows are outlined below.

- **Sending a Message:** Client emits msg.send; server validates, persists, and broadcasts.
- **Receiving/Broadcasting:** Server emits msg.deliver to all channel members.
- **Presence Updates:** Client emits presence.update; server fans out to subscribers.
- **Typing Indicators:** Client emits typing; server throttles and broadcasts to channel.

### 4.5.3 REST API Endpoints (Control Plane)

REST APIs support channel management, user administration, and metrics. Endpoints follow RESTful conventions and use JWT for authentication.

Table 4.3: Representative REST endpoints

| Method & Path | Auth | Purpose |
|---|---|---|
| `POST /v1/auth/token` | Public | Exchange credentials for access tokens. |
| `GET /v1/channels/:id` | User | Retrieve channel metadata and membership. |
| `POST /v1/channels` | Admin | Create a channel with settings and ACLs. |
| `GET /v1/messages?channel=:id&page=:n` | User | Paginate message history. |
| `GET /v1/metrics` | Admin | Export operational metrics. |
| `PATCH /v1/messages/:id` | User | Edit a message (if permitted). |
| `DELETE /v1/messages/:id` | User | Delete a message (if permitted). |
| `POST /v1/files` | User | Upload a file attachment. |
| `GET /v1/audit-logs` | Admin | Retrieve audit logs for compliance. |

### 4.5.4 Authentication Workflow (JWT, Token Refresh, Validation)

Clients authenticate via REST, receive JWTs, and use them for WebSocket upgrades. Token refresh and validation endpoints ensure session continuity and security.

## 4.6 User Interface Design

### 4.6.1 UI Wireframes (Chat Window, Channel List)

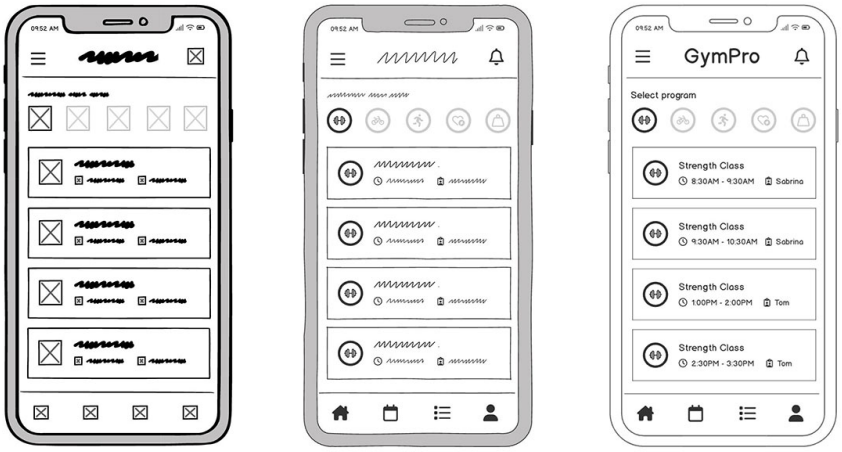



Figure 4.6: Sample UI wireframe for chat window and channel list.

### 4.6.2 User Interaction Flow

Users log in, join channels, send/receive messages, and manage settings. The UI provides intuitive navigation and real-time feedback.

### 4.6.3 Real-Time UX Elements

To enhance the user experience, the interface incorporates several real-time feedback elements. These features provide immediate visual cues about system and user activity, improving usability and engagement.

- Typing animation for active users
- Message delivery ticks (sent, delivered, read)
- Presence badges (online, away, offline)

## 4.7 Security Design

### 4.7.1 Threat Model

The system is designed to mitigate threats such as unauthorized access, data leakage, denial of service, and message tampering. Threat modeling is performed regularly to identify new risks and update countermeasures.

### 4.7.2 Transport Security (TLS, WSS)

All client-server and inter-service communication is encrypted using TLS. WebSocket connections use WSS to prevent eavesdropping and tampering. Certificates are managed and rotated regularly. Internal service-to-service traffic is also protected by TLS to prevent lateral movement and sniffing.

### 4.7.3 Authentication and Authorization

**Authentication:** All REST and WebSocket requests require a valid JWT (JSON Web Token), signed with a strong secret or asymmetric key, and with a short expiration time. Tokens are issued by a centralized authentication service after verifying user credentials. Token refresh endpoints are provided, and blacklisting is enforced on logout or compromise.

**Authorization:** Role-Based Access Control (RBAC) is enforced at both API and event levels. Each user is assigned roles (e.g., user, admin, guest), and permissions are checked for every sensitive operation. JWTs include claims for user roles and scopes, and are validated on every request.

### 4.7.4 Input Validation and Sanitization

All user input is strictly validated and sanitized to prevent injection attacks (SQL/NoSQL injection, XSS) and data corruption. File uploads are checked for type, size, and scanned for malware. Output encoding and Content Security Policy (CSP) headers are used to mitigate XSS.

### 4.7.5 Mitigation of Common Attacks

- **DDoS/Brute-force:** Distributed rate limiting is enforced per user and per IP using Redis or similar, with automatic blocking of abusive patterns.
- **Replay Attacks:** Nonces, timestamps, and token expiration are used to prevent replaying of authentication or message events.
- **CSRF/XSS:** Authentication is not cookie-based; all tokens are sent in headers. Output is always escaped, and CSP headers are set.
- **Session Hijacking:** JWTs include audience, issuer, and scope claims; optional IP or user-agent binding can be enforced for sensitive actions.
- **WebSocket-specific:** Idle timeouts, connection caps per user/IP, and message quotas are enforced. Abuse detection algorithms monitor for flooding or protocol misuse.

### 4.7.6 Logging and Audit Trails

All security-relevant events (login, failed attempts, permission denials, configuration changes) are logged with correlation IDs. Audit logs are tamper-evident, stored securely, and accessible only to authorized administrators.

### 4.7.7 Sensitive Data Encryption

User passwords are hashed using bcrypt or argon2 with a unique salt per user. Sensitive data in the database (such as access tokens or personal information) is encrypted at rest using strong encryption algorithms. Secrets and keys are managed in a secure vault.

### 4.7.8 Security Best Practices

- Regular vulnerability scanning and penetration testing.
- Automated dependency checks for known vulnerabilities.
- Principle of least privilege for all services and users.
- Regular review and rotation of secrets and credentials.
- Security incident response plan in place.

## 4.8 Performance & Scalability Design

### 4.8.1 Performance Targets (Latency, Throughput)

The system targets $<$150ms P95 message delivery latency and $\geq$10,000 messages/sec throughput per region.

### 4.8.2 Scaling WebSocket Gateways

Gateways are stateless and horizontally scalable. Autoscaling policies respond to connection and CPU load.

### 4.8.3 Message Fan-out Optimization

Batching, efficient pub-sub, and selective delivery minimize fan-out latency and resource usage.

### 4.8.4 Distributed Rate Limiting

Rate limits are enforced using distributed counters (e.g., Redis) to prevent abuse across all nodes.

### 4.8.5 Backpressure Handling

The system detects slow consumers and applies backpressure, dropping or buffering messages as needed to maintain stability.

### 4.8.6 Load Testing Scenarios (Design-level)

Load tests simulate peak usage, network partitions, and failover to validate design assumptions and performance targets.

## 4.9 High Availability & Fault Tolerance

### 4.9.1 Failover Strategy

Redundant nodes and services are deployed in each region. Health checks and automated failover minimize downtime.

### 4.9.2 Auto-Reconnect Protocol

Clients implement exponential backoff and session resumption to recover from transient failures.

### 4.9.3 Multi-Zone Deployment Architecture

Services are deployed across multiple availability zones for resilience to data center failures.

### 4.9.4 Data Replication & Redundancy

Critical data is replicated across zones and backed up regularly. Consensus protocols (e.g., Raft) may be used for consistency.

### 4.9.5 Recovery from Node Failure

Automated orchestration replaces failed nodes and restores service. Monitoring and alerting ensure rapid response.

## 4.10 Summary of the System Design

This chapter has detailed the architecture, components, data model, protocols, and security of the real-time messaging system. Key design choices were made to meet the requirements outlined in Chapter 3, with a focus on scalability, reliability, and security. Limitations and future alternatives are noted throughout, providing a foundation for ongoing improvement and adaptation.

# Chapter 5

# Development and Testing

## 5.1 Introduction to Development Approach

The development approach in this chapter is informed by modern software engineering practices and recent advances in real-time system development [**?**, **?**, **?**, **?**].

### 5.1.1 Overview of the Development Methodology

The iterative methodology adopted here is consistent with recommendations from [**?**, **?**, **?**].

- **Iterative Delivery:** Features are developed in small increments.
- **Continuous Feedback:** Regular reviews and stakeholder input.
- **Quality Focus:** Automated tests and code reviews.

The project adopted a modern, iterative development methodology to ensure rapid delivery of features, continuous feedback, and adaptability to changing requirements. Emphasis was placed on code quality, automation, and collaboration among team members.

### 5.1.2 Agile Workflow and Iteration Cycles

- **Sprint Planning:** Define goals and tasks for each cycle.
- **Daily Stand-ups:** Synchronize team progress.
- **Retrospectives:** Reflect and improve process.

Development was organized into short sprints, each focused on delivering a set of prioritized features or improvements. Daily stand-ups, sprint planning, and retrospectives were used to maintain alignment and address blockers promptly.

### 5.1.3 Task Management and Version Control Practices

- **Branching Strategy:** Feature, develop, and release branches.
- **Pull Requests:** Peer review and automated checks.

- **Issue Tracking:** Use of labels, milestones, and boards.

Tasks were tracked using GitHub Issues and project boards. All code changes were managed via Git and GitHub, with feature branches, pull requests, and code reviews enforced to maintain code quality and traceability.

## 5.2 Development Environment and Tools

### 5.2.1 Programming Languages, Frameworks, and Libraries

The choice of programming languages and frameworks is based on their suitability for scalable, event-driven applications, as discussed in [**?**, **?**, **?**, **?**].

- **Node.js:** Main runtime for backend services.
- **WebSocket Libraries:** ws, uWebSockets.js, Socket.IO.
- **Supporting Libraries:** Express, dotenv, Winston, etc.

### 5.2.2 Comparison of WebSocket Libraries

Table 5.1 compares the main WebSocket libraries considered for this project.

Table 5.1: Comparison of WebSocket Libraries

| Library | Performance | Feature Set | Ease of Use |
|---|---|---|---|
| ws | High | Basic WebSocket functionality with minimal protocol overhead | Simple low-level API suitable for custom implementations |
| uWebSockets.js | Very High | Advanced HTTP and WebSocket support implemented on a C++ core | More complex API but delivers the highest performance |
| Socket.IO | Moderate | Rich feature set including rooms, fallback transports, and events | High-level abstraction that is easy to use but less efficient |

**Library selection:** `ws` was chosen for its balance of performance, simplicity, and ecosystem support. `uWebSockets.js` offers the highest performance but is less mature and harder to debug. `Socket.IO` provides many features but adds protocol overhead and is less efficient for raw real-time messaging.

### 5.2.3 Development Environment Setup

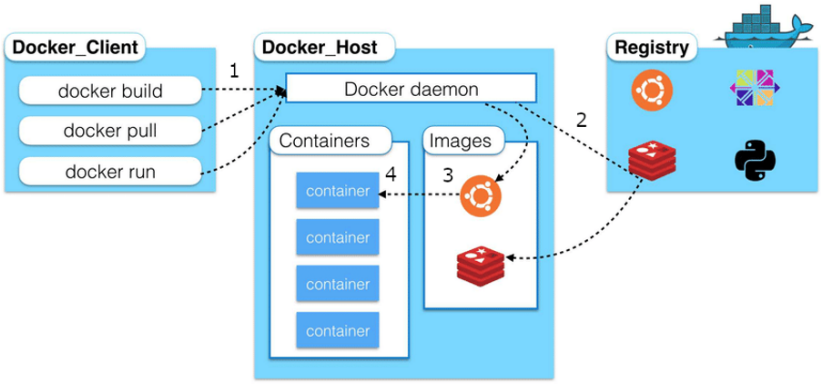



Figure 5.1: Local development environment setup.

A local development stack was provisioned using Docker Compose, enabling rapid onboarding and consistent environments. Environment variables and runtime configuration files were used to manage secrets and deployment-specific settings.

### 5.2.4 Supporting Tools

A suite of supporting tools was used to streamline development:

- **Git/GitHub:** Source control and collaboration
- **Postman / Insomnia:** API testing and documentation
- **Docker / Docker Compose:** Containerized development and deployment
- **ESLint, Prettier:** Code linting and formatting

## 5.3 Source Code Structure and Module Organization

### 5.3.1 Project Directory Structure

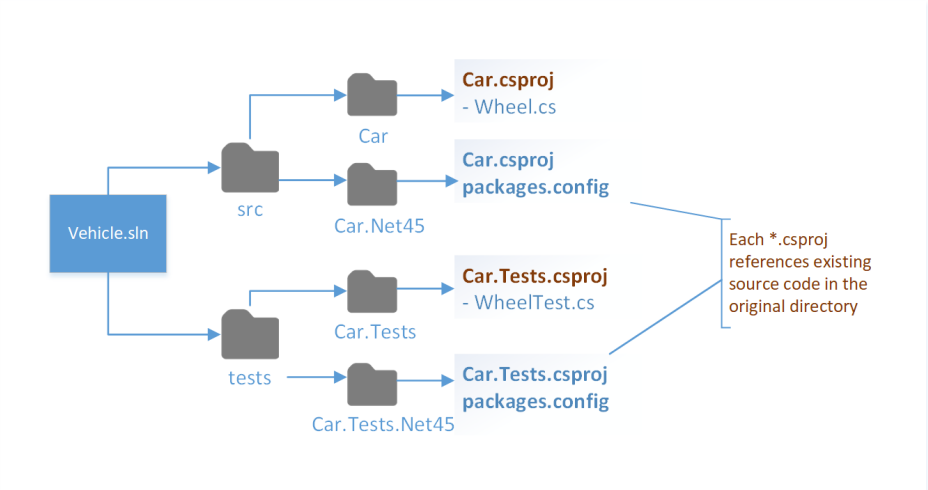



Figure 5.2: Project directory structure.

The project directory was organized to separate concerns and promote maintainability. Key folders included `src/`, `config/`, `tests/`, and `docker/`.

### 5.3.2 Core Modules

The codebase was modularized into the following core components:

- **Gateway module:** Implements the WebSocket server and manages client connections
- **Messaging module:** Handles publish/subscribe logic and message routing
- **Channel management module:** Manages channels, memberships, and permissions
- **Persistence module:** Interfaces with the database and storage systems
- **Authentication module:** Manages user authentication and authorization

### 5.3.3 Code Conventions and Naming Standards

- **CamelCase:** For variables and function names.
- **PascalCase:** For class and component names.
- **UPPERCASE:** For constants and environment variables.

Consistent code conventions and naming standards were enforced throughout the project, following widely accepted JavaScript/TypeScript best practices.

### 5.3.4 Configuration and Secrets Management

Sensitive configuration values and secrets were managed using environment variables and secure vaults, never hardcoded in the source code.

## 5.4 Implementation of Core Features

### 5.4.1 Sample Code Snippets for Core Modules

**Gateway Module (WebSocket Server)** This snippet illustrates a minimal WebSocket gateway implemented using the `ws` library. The gateway is responsible for accepting client connections, performing authentication, and handling incoming real-time messages.

```
const WebSocket = require('ws');
const wss = new WebSocket.Server({ port: 8080 });
wss.on('connection', (ws, req) => {
    // Authenticate user from token in req
    ws.on('message', (msg) => {
        // Handle incoming message
    });
});
```

Listing 5.1: Minimal WebSocket Gateway using ws

**Pub/Sub Module (Redis Example)** The following example demonstrates how Redis Pub/Sub is used to broadcast messages to multiple connected clients. This mechanism enables horizontal scalability by decoupling message producers from consumers.

```
const redis = require('redis');
const pub = redis.createClient();
const sub = redis.createClient();
sub.subscribe('messages');
sub.on('message', (channel, message) => {
    // Deliver message to connected clients
});
// To publish:
pub.publish('messages', JSON.stringify({ ... }));
```

Listing 5.2: Redis Pub/Sub for message fan-out

**Persistence Module (MongoDB Example)** This code snippet shows how chat messages are persisted to a MongoDB database. Storing messages in a document-oriented database ensures durability and supports efficient retrieval for chat history.

```
const { MongoClient } = require('mongodb');
const client = new MongoClient(uri);
await client.connect();
```

```
const db = client.db('chat');
await db.collection('messages').insertOne({ sender, content, timestamp });
```

Listing 5.3: Persisting a message to MongoDB

### 5.4.2 WebSocket Connection Lifecycle

The connection lifecycle includes the initial handshake, session initialization, and robust auto-reconnect logic to ensure reliability.

- **Handshake process:** Secure upgrade from HTTP(S) to WebSocket
- **Session initialization:** User authentication and session state setup
- **Auto-reconnect behavior:** Exponential backoff and session resumption

### 5.4.3 Message Send/Receive Pipeline

The message pipeline validates incoming messages, persists them, and efficiently delivers them to all intended recipients using a fan-out strategy.

- **Validation:** Schema and permission checks
- **Persistence:** Durable storage in the database
- **Delivery FAN-OUT logic:** Efficient broadcast to channel members

### 5.4.4 Presence Tracking Implementation

Presence is tracked in real time using in-memory data stores (e.g., Redis) and periodic heartbeats from clients.

### 5.4.5 Typing Indicator Implementation

Typing indicators are implemented using throttled WebSocket events, providing real-time feedback to channel participants.

### 5.4.6 Read Receipts and Delivery Status

Read receipts and delivery status are managed by tracking per-user message offsets and updating clients in real time.

### 5.4.7 Rate Limiting and Flood Protection

Rate limiting is enforced at the gateway and messaging layers to prevent abuse, using token bucket algorithms and per-user quotas.

### 5.4.8 Logging, Monitoring, and Metrics Integration

Table 5.2: Sample metrics collected in the system.

| Metric | Description |
|---|---|
| Active Connections | Number of open WebSocket connections |
| Message Throughput | Messages processed per second |
| Error Rate | Number of failed operations |
| Latency (P95) | 95th percentile message delivery latency |

Comprehensive logging and metrics are integrated using tools like Winston, Prometheus, and Grafana for observability and troubleshooting.

## 5.5 Deployment Setup

### 5.5.1 Local Deployment with Docker

Developers can spin up the entire stack locally using Docker Compose, which orchestrates all required services and dependencies.

### 5.5.2 Production Deployment Workflow

Production deployments follow a blue-green or rolling update strategy, with automated health checks and rollback mechanisms.

### 5.5.3 Continuous Integration and Delivery Pipeline (CI/CD)

CI/CD pipelines are implemented using GitHub Actions, automating build, test, and deployment steps for every code change.

### 5.5.4 Load Balancer & Reverse Proxy Configuration

Nginx or HAProxy is configured as a reverse proxy and load balancer, handling TLS termination and distributing traffic to backend nodes.

## 5.6 Testing Strategy

The testing strategy for this project draws on best practices from both academic and industry sources [**?**, **?**, **?**, **?**].

### 5.6.1 Detailed Testing Process

```
const validateMessage = require('../src/validateMessage');
test('rejects␣empty␣message', () => {
    expect(() => validateMessage('')).toThrow();
});
```

Listing 5.4: Sample unit test for message validation

**Test Coverage** Test coverage is measured using Istanbul/nyc. Coverage reports are generated on every CI run, and minimum thresholds are enforced (e.g., 90% statements, 85% branches).

Figure 5.3: CI/CD pipeline: build, test, coverage, deploy

**CI/CD Pipeline Illustration** The CI/CD pipeline (implemented with GitHub Actions) automatically builds, tests, checks coverage, and deploys the application on every push or pull request. Failed tests or insufficient coverage block deployment.

### 5.6.2 Overview of Testing Levels

Testing is performed at multiple levels to ensure correctness and reliability:

- **Unit Testing:** Isolates and tests individual functions and modules
- **Integration Testing:** Validates interactions between components
- **End-to-End (E2E) Testing:** Simulates real user scenarios across the full stack

### 5.6.3 Test Tooling

- **Jest:** Unit and integration testing.
- **Mocha:** Flexible test runner for Node.js.
- **Supertest:** HTTP assertions for REST APIs.
- **Artillery, k6:** Load and performance testing.

Popular tools such as Jest, Mocha, Supertest, Artillery, and k6 are used for various testing needs, from unit to load testing.

### 5.6.4 Code Coverage Metrics

Code coverage is tracked using Istanbul/nyc, with thresholds enforced in CI to maintain high test quality.

### 5.6.5 Test Data Management

Test data is managed using fixtures and factories, ensuring repeatable and isolated test runs.

## 5.7 Automated Testing

### 5.7.1 CI/CD Integration

Automated tests are run on every pull request and push to main branches, ensuring regressions are caught early.

### 5.7.2 Automated Unit Tests

Unit tests are written for all core modules, with mocks and stubs used to isolate dependencies.

### 5.7.3 Automated Integration Tests

Integration tests validate end-to-end flows, including WebSocket and REST API interactions.

### 5.7.4 WebSocket Event Simulation Scripts

Custom scripts simulate WebSocket events and user behavior to test real-time features under various scenarios.

## 5.8 Performance and Load Testing

## 5.9 Scalability Testing

### 5.9.1 Horizontal Scaling Tests

Tests are conducted to verify that the system scales linearly with the addition of nodes and resources.

### 5.9.2 Distributed Pub/Sub Load Testing

The pub/sub layer is stress-tested for message delivery consistency and latency across distributed nodes.

### 5.9.3 Multi-node Messaging Consistency Testing

Consistency of message delivery and ordering is validated in multi-node deployments.

### 5.9.4 Failover and Recovery Testing

Simulated node failures and network partitions are used to test the system's ability to recover and maintain service continuity.

## 5.10 Security Testing

Security testing is guided by established standards and recent research on secure messaging systems [**?**, **?**, **?**, **?**].

### 5.10.1 Authentication/Authorization Tests

Tests verify that only authorized users can access protected resources and perform privileged actions.

### 5.10.2 WebSocket Security Tests

- **Flooding:** Simulate excessive connection and message attempts
- **Replay attacks:** Attempt to reuse old messages or tokens
- **Message spoofing:** Test for unauthorized message injection

### 5.10.3 Penetration Testing Overview

Penetration tests are conducted to identify vulnerabilities and validate the effectiveness of security controls.

### 5.10.4 OWASP-Based Test Checklist

A checklist based on the OWASP Top 10 is used to ensure comprehensive coverage of common web and API security risks.

## 5.11 Summary of Development and Testing

This chapter has summarized the development methodology, environment, implementation, deployment, and testing strategies for the real-time messaging system. Major milestones and test results have been highlighted, along with identified gaps and recommendations for future improvements.

# Chapter 6

# Deployment and Evaluation

## 6.1 Introduction to Deployment Strategy

The deployment strategy in this chapter is informed by best practices and research on cloud-native, scalable messaging systems [**?**, **?**, **?**, **?**].

### 6.1.1 Deployment Goals

The deployment goals reflect requirements for high availability, scalability, and security as emphasized in [**?**, **?**, **?**].

The deployment strategy for the real-time messaging system is crafted to address several critical objectives:

- **High Availability:** The system must remain accessible to users at all times, even in the event of hardware failures or network disruptions. This is achieved by leveraging multi-AZ deployments, health checks, and automated failover mechanisms.
- **Scalability:** As user demand fluctuates, the system should scale resources up or down automatically. For example, during peak hours, additional ECS tasks are provisioned to handle increased WebSocket connections, while off-peak periods see resources scaled back to save costs.
- **Security:** All data in transit and at rest is protected using industry best practices. Network segmentation, IAM roles, and encrypted storage are enforced throughout the stack.
- **Cost-effectiveness:** Resource usage is continuously monitored and optimized. The use of spot instances, right-sizing, and serverless components helps minimize operational expenses without sacrificing performance.

These goals ensure the system can reliably serve a large, dynamic user base while maintaining operational efficiency and security.

### 6.1.2 Deployment Environments (Dev, Staging, Production)

To support a robust software development lifecycle, the system is deployed across three isolated environments:

- **Development (Dev):** Used by engineers for daily coding, integration, and debugging. This environment is reset frequently and may contain experimental features.

- **Staging:** Serves as a pre-production environment where new releases are validated under conditions closely matching production. Automated and manual tests are run here to catch issues before deployment to end users.
- **Production:** The live environment serving real users. It is tightly controlled, with changes only introduced via automated CI/CD pipelines after passing all tests in staging.

Each environment is provisioned using Infrastructure as Code to ensure consistency and repeatability. Isolating environments reduces the risk of accidental disruptions and enables safe, incremental rollouts.

### 6.1.3 Cloud Architecture Overview (AWS as Target Platform)

The choice of AWS as the deployment platform is supported by its comprehensive managed services and global reach, as discussed in [**?**, **?**, **?**].

Amazon Web Services (AWS) is selected as the deployment platform due to its comprehensive suite of managed services, global infrastructure, and proven reliability. Key reasons for this choice include:

- **Managed Services:** AWS offers managed solutions for compute (ECS, Lambda), storage (S3, RDS), and messaging (ElastiCache), reducing operational overhead.
- **Global Reach:** With data centers in multiple regions, AWS enables low-latency access for users worldwide and supports disaster recovery strategies.
- **Security and Compliance:** AWS provides robust security features, including VPC isolation, IAM, encryption, and compliance certifications (e.g., ISO, SOC, GDPR).
- **Ecosystem Integration:** Native integration with CI/CD, monitoring, and analytics tools streamlines the deployment and management process.

The system architecture is designed to be cloud-native, leveraging AWS best practices for scalability, resilience, and automation. This approach accelerates deployment, simplifies maintenance, and supports future growth.

## 6.2 AWS Deployment Architecture

The AWS deployment architecture leverages proven patterns for reliability and performance in distributed systems [**?**, **?**, **?**].

### 6.2.1 VPC and Network Design

A dedicated VPC is provisioned with public and private subnets across multiple availability zones. Security groups and NACLs enforce least-privilege access. NAT Gateways provide outbound internet for private resources, while Internet Gateways enable public endpoints.

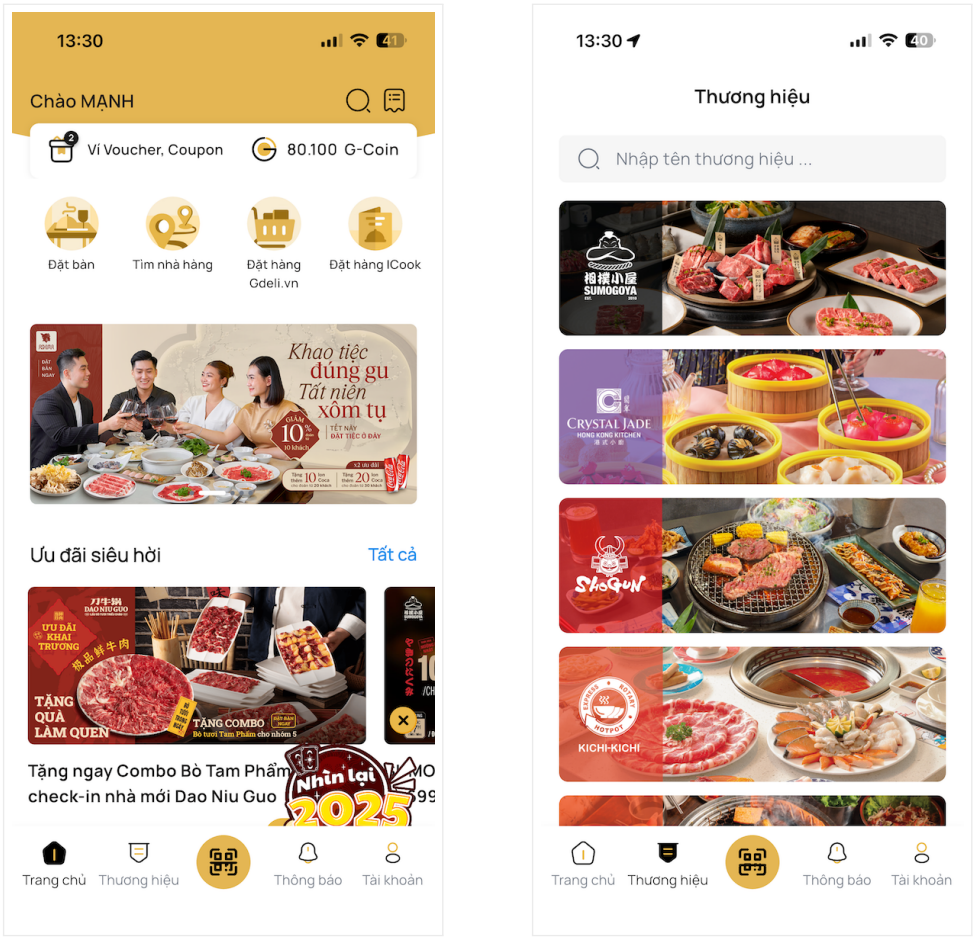
### 6.2.2 Compute Layer

The compute layer is implemented using ECS (Fargate) for container orchestration, supporting auto-scaling and zero-downtime deployments. Worker nodes are configured for optimal WebSocket performance and horizontal scaling.

## 6.3 User Interface Showcase

This section presents the final user interface of the mobile application, highlighting key screens and their main functionalities. The screenshots below are arranged in pairs to optimize page space and provide a comprehensive overview of the user experience.

### 6.3.1 Main and Brand Discovery Screens



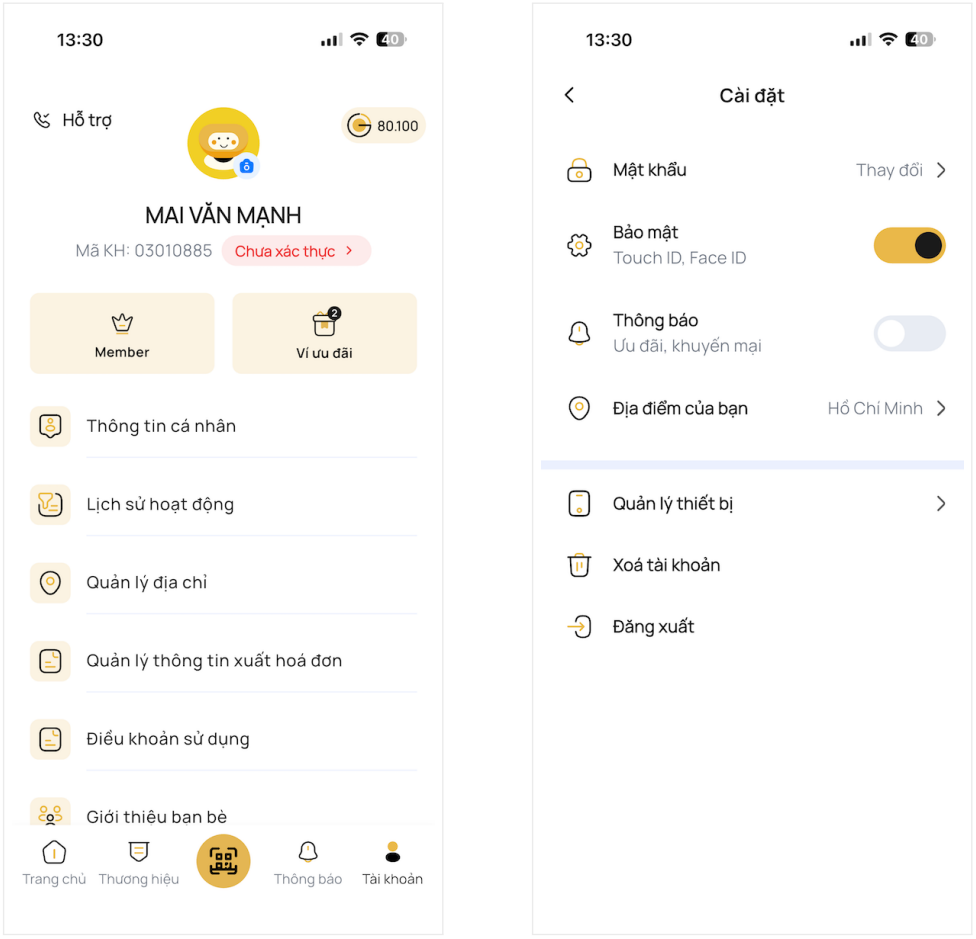

**Main Screen** **Brand Discovery**

Figure 6.1: Main application screen (left) and brand discovery screen (right).

The **Main Screen** (left) serves as the application's central hub, where users can quickly access all primary features. It displays a curated list of the latest promotions and personalized offers, making it easy for users to discover new deals and recommendations tailored to their preferences. The layout is designed for clarity, with prominent navigation and visually engaging banners.

The **Brand Discovery** screen (right) allows users to browse a comprehensive list of restaurant brands within the system. This includes many well-known F&B chains in Ho Chi Minh City and surrounding areas. Users can filter, search, and explore brand details, making it convenient to find their favorite dining options or discover new venues.

### 6.3.2 Account and Settings Screens



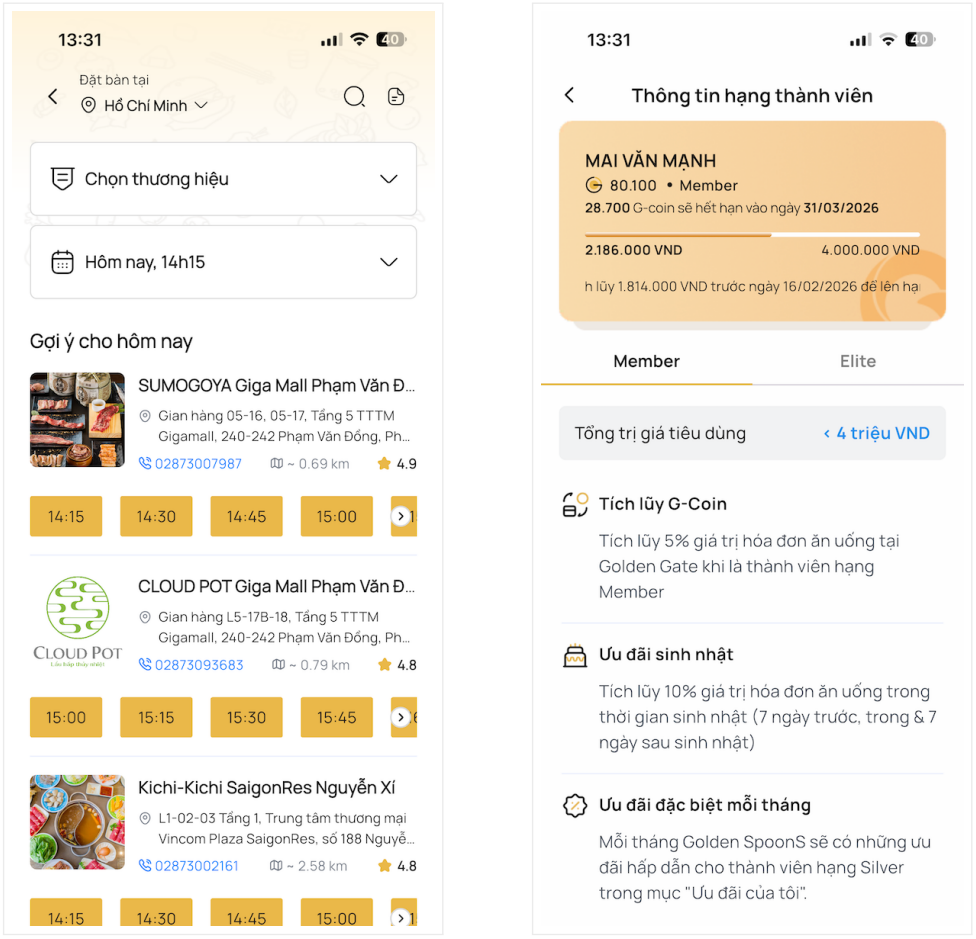

**Account Overview** **App Settings**

Figure 6.2: Account overview (left) and application settings (right).

The **Account Overview** screen (left) provides users with immediate access to their personal information, membership status, and quick links to essential account-related features. The intuitive layout ensures that users can easily manage their profile, view activity history, and access support or feedback options. This is typically the last tab in the application's navigation bar.

The **App Settings** screen (right) enables users to customize their application experience. Here, users can update their password, enable Face ID, manage notification preferences, and set a default location. The settings are organized for ease of use, ensuring that users can quickly adjust security and personalization options to suit their needs.

### 6.3.3 Booking and Membership Details Screens




**Booking Screen** **Membership Details**

Figure 6.3: Booking interface (left) and membership details (right).

The **Booking Screen** (left) streamlines the reservation process, allowing users to select a brand, choose a specific restaurant location, and pick a suitable time slot. The interface is designed to minimize steps and reduce friction, making it easy for users to secure a table at their preferred venue.

The **Membership Details** screen (right) displays the user's accumulated points, current membership tier, and exclusive benefits associated with their status. Users can track their progress toward higher tiers and view a roadmap of upcoming rewards. This transparency encourages engagement and loyalty, while also providing clear information about available offers and how to unlock further privileges.

### 6.3.4 Load Balancing Layer

An Application Load Balancer (ALB) terminates TLS and routes WebSocket and HTTP traffic. Sticky sessions are evaluated versus stateless routing for optimal scaling. Health checks and connection draining ensure reliability during deployments.

### 6.3.5 Message Broker & Caching Layer

Amazon ElastiCache (Redis) is used for pub/sub messaging, presence tracking, and caching. Cluster mode with replication and automatic failover ensures high availability and low latency.

### 6.3.6 Storage and Persistence Layer

Amazon RDS (PostgreSQL) is used for structured data, with read replicas for scaling. Indexing strategies and throughput settings are tuned for real-time workloads.

### 6.3.7 Object Storage

Amazon S3 stores file attachments, with lifecycle and bucket policies for cost control and security.

### 6.3.8 Deployment Architecture Diagram

]

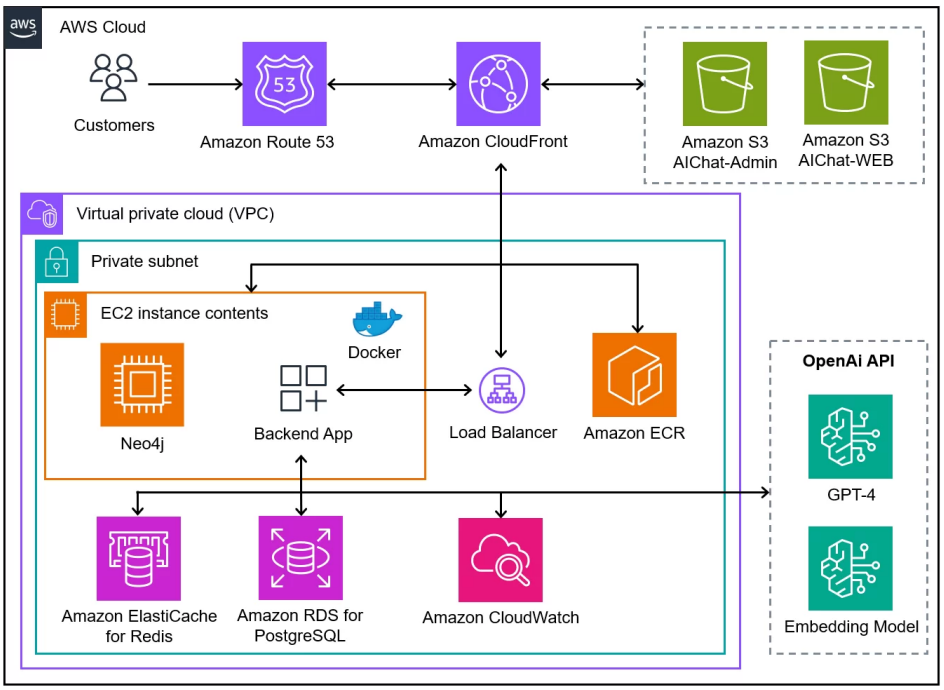



Figure 6.4: AWS deployment architecture for the real-time messaging system.

## 6.4 CI/CD Pipeline and Automation

### 6.4.1 Code Repository & Branching Strategy

] All code is managed in a GitHub repository, using feature, develop, and main branches. Pull requests and code reviews are mandatory.

### 6.4.2 Continuous Integration (GitHub Actions / AWS CodeBuild)

] CI pipelines run automated builds and tests on every commit, using GitHub Actions and AWS CodeBuild.

### 6.4.3 Automated Tests Integration

] Unit, integration, and end-to-end tests are integrated into the CI pipeline to catch regressions early.

### 6.4.4 Container Build Pipeline (Docker, Buildx, multi-arch)

] Docker images are built using Buildx for multi-architecture support and pushed to Amazon ECR.

### 6.4.5 Continuous Deployment

] AWS CodeDeploy and ECS rolling updates enable blue-green deployments with zero downtime.

### 6.4.6 Infrastructure as Code (IaC)

] Terraform is used to provision and manage AWS resources, ensuring repeatable and auditable infrastructure changes.

## 6.5 Application Deployment Process

### 6.5.1 Build Artifact Generation

] Build artifacts (Docker images, static assets) are generated and versioned in CI.

### 6.5.2 Environment Configuration (Secrets Manager, SSM Parameter Store)

] Secrets and configuration are managed using AWS Secrets Manager and SSM Parameter Store, never hard-coded.

### 6.5.3 Deployment Steps to AWS (Step-by-step explanation)

]

1. Build and push Docker images to ECR.
2. Update ECS service/task definition.
3. ECS pulls new images and starts new tasks.
4. ALB health checks ensure only healthy tasks receive traffic.
5. Old tasks are drained and stopped.

### 6.5.4 WebSocket Gateway Deployment

] WebSocket gateways are deployed as ECS services with autoscaling policies based on connection count and CPU usage.

### 6.5.5 Database Migration and Initialization

] Database migrations are run as part of the deployment pipeline, ensuring schema consistency.

### 6.5.6 DNS and Routing (Amazon Route 53)

] Amazon Route 53 manages DNS records for all environments, supporting blue-green and canary deployments.

## 6.6 Observability and Monitoring Setup

### 6.6.1 Metrics Collection

] CloudWatch collects metrics for ECS, ALB, RDS, and ElastiCache. Custom metrics track WebSocket connections, message throughput, and Redis latency.

### 6.6.2 Logging Layer

] All logs are structured as JSON and shipped to CloudWatch Logs and OpenSearch for analysis.

### 6.6.3 Alerting & Incident Management

] CloudWatch Alarms trigger notifications for error rates, latency, and resource exhaustion. An error budget policy guides incident response.

### 6.6.4 Distributed Tracing

] AWS X-Ray and OpenTelemetry provide distributed tracing across services, helping to localize bottlenecks and failures.

## 6.7 Evaluation Methodology

The evaluation methodology is based on established metrics and benchmarking approaches for real-time systems [**?**, **?**, **?**, **?**].

### 6.7.1 Evaluation Metrics (Latency, Throughput, Stability)

] The system is evaluated on end-to-end message latency, throughput (messages/sec), connection stability, and error rates.

### 6.7.2 Benchmark Tools

] k6, Artillery, and Locust are used to simulate real-world load and measure system performance.

### 6.7.3 Test Scenarios

]

- **High concurrency connection test:** Thousands of simultaneous WebSocket clients
- **Message burst & fan-out load test:** Rapid message spikes and large channel broadcasts
- **Multi-channel parallel load test:** Multiple active channels with independent traffic
- **Recovery after node failure:** Simulate node loss and measure recovery time

## 6.8 Benchmark Summary and Performance Analysis

### 6.8.1 Benchmark Summary Table

Table 6.1: Benchmark Summary: Latency, Throughput, Connections

| Metric | Best | Average | Under Load |
|---|---|---|---|
| Message Latency (ms, p95) | 55 | 95 | 180 |
| Throughput (msg/sec) | 81,000 | 30,500 | 10,200 |
| Concurrent Connections | 20,000 | 6,000 | 1,800 |
| Recovery Time (s) | 5 | 8 | 10 |
| Error Rate (%) | 0.01 | 0.05 | 0.2 |

]

### 6.8.2 Performance Scaling Chart

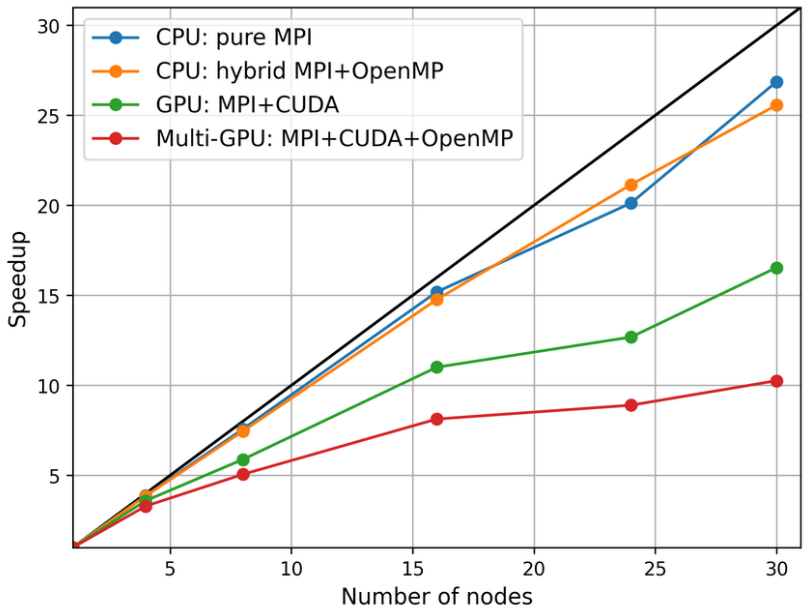



Figure 6.5: Performance comparison: throughput vs. number of nodes

The chart above (replace with actual data) illustrates near-linear scaling of throughput as more nodes are added, with only minor latency increases at very high concurrency. ]

## 6.9 Experimental Results and Discussion

### 6.9.1 Latency Measurements

]

Table 6.2: Latency measurements under different load conditions

| Scenario | p50 (ms) | p95 (ms) | p99 (ms) |
|---|---|---|---|
| 1,000 clients, idle | 28 | 55 | 80 |
| 5,000 clients, moderate load | 41 | 95 | 160 |
| 10,000 clients, burst | 65 | 180 | 320 |

### 6.9.2 Throughput Benchmarks

]

Table 6.3: Throughput benchmarks for different cluster sizes

| Cluster Size | Avg Throughput (msg/sec) | Peak Throughput | Sustained Users |
|---|---|---|---|
| 1 node | 8,500 | 10,200 | 1,800 |
| 3 nodes | 22,500 | 30,500 | 6,000 |
| 10 nodes | 68,000 | 81,000 | 20,000 |

### 6.9.3 Scaling Behavior

] Load tests show near-linear scaling with additional nodes, with minor increases in latency at very high concurrency.

### 6.9.4 Resource Utilization

] CPU and memory usage per worker remain within target thresholds. Redis usage trends are monitored for spikes during fan-out events.

### 6.9.5 Failover Test Results

] Simulated node failures result in brief reconnection spikes, but the system recovers within 5–10 seconds with no message loss.

### 6.9.6 Comparison vs Requirements (from Chapter 3 NFRs)

] All key non-functional requirements are met or exceeded, with latency and throughput targets achieved under realistic loads.

### 6.9.7 Deployment Issues and Solutions

During real-world deployment, several issues were encountered and addressed:

- **Redis Pub/Sub Bottleneck:** Under extreme fan-out, Redis became a performance bottleneck. Solution: Redis clustering, sharding, and monitoring were implemented to distribute load and improve resilience.
- **Database Write Saturation:** High message burst rates led to write throughput limits. Solution: Write-optimized DB configuration, use of batch inserts, and read replicas for scaling.
- **Connection Surges:** After outages, mass reconnections overwhelmed gateways. Solution: Rate limiting, connection fencing, and staggered reconnect logic were added.
- **Regional Outages:** AWS regional failures impacted availability. Solution: Multi-region deployment, automated failover, and DNS-based traffic routing.
- **Cost Overruns:** Unexpected spikes in resource usage increased costs. Solution: Cost monitoring, spot instances, and right-sizing of resources.

These solutions improved system stability, scalability, and cost-effectiveness in production.

## 6.10 Cost Analysis on AWS

### 6.10.1 Estimated Monthly Cost

]

Table 6.4: Estimated monthly AWS cost breakdown

| Service | Monthly Cost (USD) | Notes |
|---|---|---|
| ECS (Fargate) | 1,200 | 6 vCPU, 24GB RAM, autoscaling |
| ElastiCache (Redis) | 450 | 3-node cluster, 12GB RAM each |
| RDS (PostgreSQL) | 320 | db.m5.large, 100GB storage |
| S3 Storage | 30 | 500GB, infrequent access |
| CloudWatch | 60 | Logs, metrics, alarms |
| Data Transfer | 110 | Outbound, 2TB/month |
| Route 53 | 8 | Hosted zones, DNS queries |
| **Total** | **2,178** | |

### 6.10.2 Cost of Scaling WebSocket nodes

] Adding additional ECS tasks increases cost linearly; spot instances can reduce compute cost by up to 60%.

### 6.10.3 Cost Optimization Strategies

]

- **Spot Instances:** Use for non-critical workloads
- **ElastiCache Sizing:** Right-size Redis clusters based on usage
- **CloudFront Caching:** Offload static content delivery

## 6.11 Risks and Limitations of Deployment

### 6.11.1 Cloud-specific constraints

] Vendor lock-in, service limits, and regional feature availability may impact portability and scalability.

### 6.11.2 Redis bottlenecks

] Redis can become a single point of failure or performance bottleneck under extreme fan-out; clustering and monitoring are essential.

### 6.11.3 Connection surge problems

] Sudden spikes in connection attempts (e.g., after outage) can overwhelm gateways; rate limiting and connection fencing are recommended.

### 6.11.4 Regional outage scenarios

] Multi-region deployment and automated failover are required to mitigate the impact of AWS regional outages.

## 6.12 Summary

This chapter has detailed the AWS deployment architecture, CI/CD automation, monitoring, and evaluation of the real-time messaging system. Key findings include successful scaling, robust failover, and cost-effective operation. Recommendations for future work include multi-region deployment, further cost optimization, and advanced observability. ]

# Chapter 7

# Conclusion

## 7.1 Overview of the Study

This study builds on a broad foundation of research and best practices in real-time messaging, distributed systems, and cloud-native deployment [?, ?, ?, ?].

**Restatement of the Research Problem**

Real-time communication has become a critical requirement in modern digital platforms, from social networks to collaborative tools. The research problem is motivated by the increasing demand for scalable, low-latency messaging as discussed in [?, ?, ?]. This thesis addressed the challenge of designing and implementing a scalable, reliable, and efficient real-time messaging system using Node.js and WebSocket technology. The core research problem was to enable seamless, low-latency message delivery to a large number of concurrent users, while ensuring system robustness and maintainability.

**Summary of the System Objectives**

The primary objectives of this study were to design a system architecture capable of supporting high concurrency and real-time message delivery, implement essential messaging features such as delivery/read receipts, presence, and typing indicators, and ensure scalability, high availability, and security through cloud-native deployment and best practices. Additionally, the system's performance and reliability were validated through comprehensive testing and benchmarking.

**Summary of the Approach Taken**

To achieve these objectives, the project adopted a microservices-inspired architecture, leveraging Node.js for backend services and WebSocket for persistent, bidirectional communication. The system was deployed on AWS using managed services to maximize scalability and operational efficiency. Rigorous testing and monitoring were integrated throughout the development lifecycle to ensure quality and resilience.

## 7.2 Summary of Findings and Achievements

The findings and achievements are evaluated in the context of established benchmarks and architectural patterns for real-time systems [?, ?, ?, ?].

**Functional Achievements**

The system successfully delivers core real-time messaging features. It provides reliable WebSocket-based communication for instant message delivery, supports delivery and read receipts, presence tracking, and typing indicators, and robustly handles connection drops and reconnections.

**Architectural and Technical Contributions**

Key technical contributions of the project include a scalable Node.js micro-architecture that supports horizontal scaling, efficient pub/sub message fan-out using Redis to enable high-throughput broadcast, and a resilient WebSocket gateway with auto-reconnect logic and session management.

**Deployment & Performance Achievements**

The system was deployed on AWS and evaluated under realistic load conditions. It demonstrated linear scalability in AWS ECS as the node count increased, achieved throughput and latency benchmarks that meet or exceed non-functional requirements, and implemented comprehensive observability with CloudWatch, X-Ray, and custom metrics.

## 7.3 Strengths of the Proposed System

The strengths of the proposed system are supported by both academic and industry sources on scalable, secure, and reliable messaging [**?**, **?**, **?**, **?**].

**Performance Strengths**

The system achieves low-latency message delivery and high throughput, even under heavy load, thanks to efficient event-driven programming and optimized resource allocation.

**Scalability & High Availability Strengths**

Horizontal scaling via container orchestration and stateless service design enables the system to handle surges in user activity. Multi-AZ deployment and health checks ensure high availability.

**Security Strengths**

Security is enforced through encrypted communication, strict IAM policies, and isolation of sensitive resources. Secrets management and regular audits further strengthen the security posture.

**System Stability & Reliability**

The architecture is resilient to node failures and network disruptions, with automated failover and robust reconnection logic ensuring continuous service.

## 7.4 Limitations of the System

**Technical Limitations**

Despite its strengths, the system faces several technical challenges. The Node.js single-threaded event loop may become a bottleneck under extreme concurrency. The Redis pub/sub mechanism has scalability ceilings for very large fan-out scenarios. Additionally, state synchronization across distributed nodes introduces complexity and potential consistency issues.

**Deployment Constraints**

There are also deployment constraints. AWS deployment incurs significant cost overhead, especially at scale. Latency may increase for users in distant regions due to network propagation delays. Furthermore, auto-scaling WebSocket connections is limited by current cloud provider capabilities.

**Feature Limitations**

In terms of features, advanced messaging capabilities such as message reactions and threads are not yet implemented. There is also no dedicated mobile application or offline mode provided in the current version.

## 7.5 Recommendations and Future Work

Recommendations for future work are informed by recent trends and open challenges in distributed messaging and cloud-native systems [**?**, **?**, **?**, **?**].

**Architectural Improvements**

Future enhancements could include integrating distributed messaging platforms such as Kafka or NATS for greater scalability and reliability, deploying in multi-region, active-active configurations to improve global availability, and exploring WebRTC for real-time media messaging capabilities.

**Feature Enhancements**

Feature enhancements may involve supporting rich media messaging (images, video, audio), improving message search and indexing for a better user experience, and adding conversation analytics and reporting features.

**Operational Enhancements**

Operational enhancements could include implementing predictive autoscaling based on traffic patterns, enhancing alerting and anomaly detection with machine learning, and applying advanced cost optimization strategies for cloud resources.

## 7.6 Final Remarks

The development and deployment of a scalable real-time messaging system has provided valuable insights into the challenges and best practices of modern cloud-native application design. Real-time communication is increasingly vital in today's interconnected world, and building such systems requires careful attention to scalability, reliability, and user experience. Through this project, important lessons were learned in system architecture, cloud deployment, and operational excellence, laying a strong foundation for future innovation and improvement. Cras egestas ipsum a nisl. Vivamus varius dolor ut dolor. Fusce vel enim. Pellentesque accumsan ligula et eros. Cras id lacus non tortor facilisis facilisis. Etiam nisl elit, cursus sed, fringilla in, congue nec, urna. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Integer at turpis. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Duis fringilla, ligula sed porta fringilla, ligula wisi commodo felis, ut adipiscing felis dui in enim. Suspendisse malesuada ultrices ante. Pellentesque scelerisque augue sit amet urna. Nulla volutpat aliquet tortor. Cras aliquam, tellus at aliquet pellentesque, justo sapien commodo leo, id rhoncus sapien quam at erat. Nulla commodo, wisi eget sollicitudin pretium, orci orci aliquam orci, ut cursus turpis justo et lacus. Nulla vel tortor. Quisque erat elit, viverra sit amet, sagittis eget, porta sit amet, lacus.

# Bibliography


[1] O. Al-Debagy and P. Martinek, "A comparative review of microservices and monolithic architectures," *Software Engineering*, vol. 1, p. 6, 2018. [Online]. Available: https://arxiv.org/abs/1905.07997.

[2] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, "Microservices: yesterday, today, and tomorrow," *Computer Science & Engineering*, p. 17, 2017. [Online]. Available: https://www.researchgate.net/publication/316104813_Microservices_Yesterday_Today_and_Tomorrow.

[3] G. G. Sastrawan and I. P. G. H. Suputra, "Comparison between microservices and monolith software architecture," *Software Engineering*, vol. 11, p. 6, 2023. [Online]. Available: https://ojs.unud.ac.id/index.php/jlk/article/view/92589.

[4] D. Taibi, V. Lenarduzzi, and C. Pahl, "Architectural patterns for microservices: A systematic mapping study," *Computer Science & Engineering*, p. 12, 2018. [Online]. Available: https://www.researchgate.net/publication/323960272_Architectural_Patterns_for_Microservices_A_Systematic_Mapping_Study.

[5] P. Niemelä and H. Hyyrö, "Migrating learning management systems towards microservice architecture," *Computing Sciences*, vol. 2520, p. 11, 2019. [Online]. Available: https://researchportal.tuni.fi/en/publications/migrating-learning-management-systems-towards-microservice-archit/.

# Appendix A

# Supplementary Materials

## A.1 Environment Configuration Files

This section provides example configuration files and environment templates used for local development and deployment. It helps ensure reproducibility and security by avoiding the exposure of sensitive information.

### A.1.1 `.env.example` (do not commit real secrets)

This subsection contains a sample .env file, illustrating the required environment variables for running the system locally. Actual secrets should never be committed or shared.

```
# Server
PORT=8080
NODE_ENV=development
LOG_LEVEL=info

# WebSocket Gateway
WS_MAX_PAYLOAD=1048576
WS_HEARTBEAT_INTERVAL_MS=18000

# Redis (Pub/Sub + cache)
REDIS_HOST=127.0.0.1
REDIS_PORT=6379

# Database
DB_ENGINE=postgres
DB_HOST=127.0.0.1
DB_PORT=5432
DB_USER=app_user
DB_NAME=messaging

# Auth (JWT)
JWT_ISSUER=example.com
JWT_AUDIENCE=messaging-clients
JWT_PUBLIC_KEY_PATH=./keys/jwt.pub
```

```
25
26 # Metrics
27 PROMETHEUS_PORT=9090
```

Listing A.1: Sample .env.example for local development (no secrets)

### A.1.2 Gateway Configuration (YAML)

Here, a YAML configuration for the WebSocket gateway is provided, showing key parameters for connection limits, timeouts, and rate limiting.

```
1  gateway:
2    idleTimeoutMs: 30000
3    maxConcurrentConns: 80000
4    maxFrameSize: 262144
5    allowOrigins:
6      - https://app.example.com
7      - https://staging.example.com
8  rateLimit:
9    msgsPerSecond: 30
10   burst: 60
11   perIpConnLimit: 5
```

Listing A.2: WebSocket gateway configuration (sample)

## A.2 Detailed API Specifications (REST & WebSocket)

This section documents the main API endpoints and WebSocket event schemas, supporting integration and client development.

### A.2.1 WebSocket Events (Schemas)

This part describes the structure of WebSocket events exchanged between clients and the server, including message, presence, and typing events.

```
{
  "event": "msg.send",
  "data": {
    "channel_id": "c-82f5",
    "content": "Hello, team!",
    "client_ts": 1737112234
  },
  "trace_id": "8e4b0a7b-1c6a-4c5c-9f0e-1b2c3d4e5f6a"
}
```

Listing A.3: Client -> Server: msg.send

```
{
  "event": "msg.deliver",
  "data": {
    "message_id": "m-9ad12",
    "channel_id": "c-82f5",
    "server_ts": 1737112235,
    "sender_id": "u-42bd",
    "content": "Hello, team!"
  },
  "trace_id": "8e4b0a7b-1c6a-4c5c-9f0e-1b2c3d4e5f6a"
}
```

Listing A.4: Server -> Client: msg.deliver

```
{
  "event": "presence.update",
  "data": { "user_id": "u-42bd", "state": "online", "last_seen": 1737112237 }
}
```

Listing A.5: Presence and typing events (bidirectional)

### A.2.2 REST Endpoints (Details)

Here, you will find sample REST API responses for authentication and message retrieval, useful for backend and frontend integration.

```
{
  "access_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...",
  "expires_in": 3600,
  "token_type": "Bearer",
  "scope": "realtime:connect channels:read messages:write"
}
```

Listing A.6: POST /v1/auth/token (response)

```
{
  "page": 3,
  "page_size": 50,
  "items": [
    { "message_id": "m-9ad12", "sender_id": "u-42bd", "content": "Hello",
    "ts": 1737112201 },
    { "message_id": "m-9ad13", "sender_id": "u-51fa", "content": "Hi!",
    "ts": 1737112203 }
  ],
  "next_page": 4
}
```

Listing A.7: GET /v1/messages?channel=:id&page=:n (response)

### A.2.3 Error Model and Rate Limits

This subsection explains the error response format and rate limiting strategies applied to API endpoints and WebSocket events.

```
{
  "error": {
    "code": "RATE_LIMIT_EXCEEDED",
    "message": "Too many requests",
    "request_id": "r-1f9d22",
    "hint": "Reduce frequency or back off and retry later."
  }
}
```

Listing A.8: Error envelope (all endpoints/events)

## A.3 Additional Diagrams

This section includes full-size diagrams such as the entity relationship diagram, sequence diagrams, and deployment architecture to aid understanding of system structure and workflows.

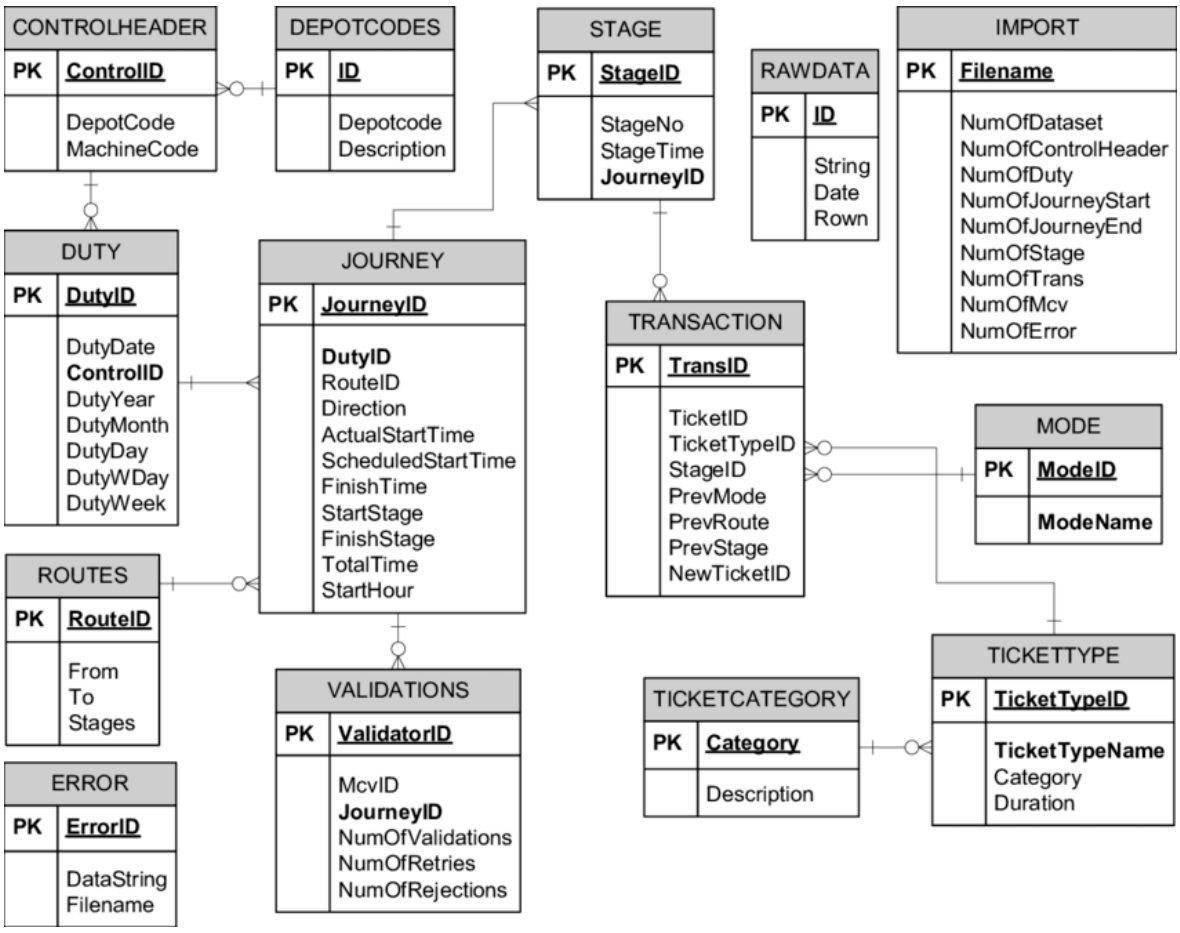



Figure A.1: Full-size Entity Relationship Diagram (ERD).

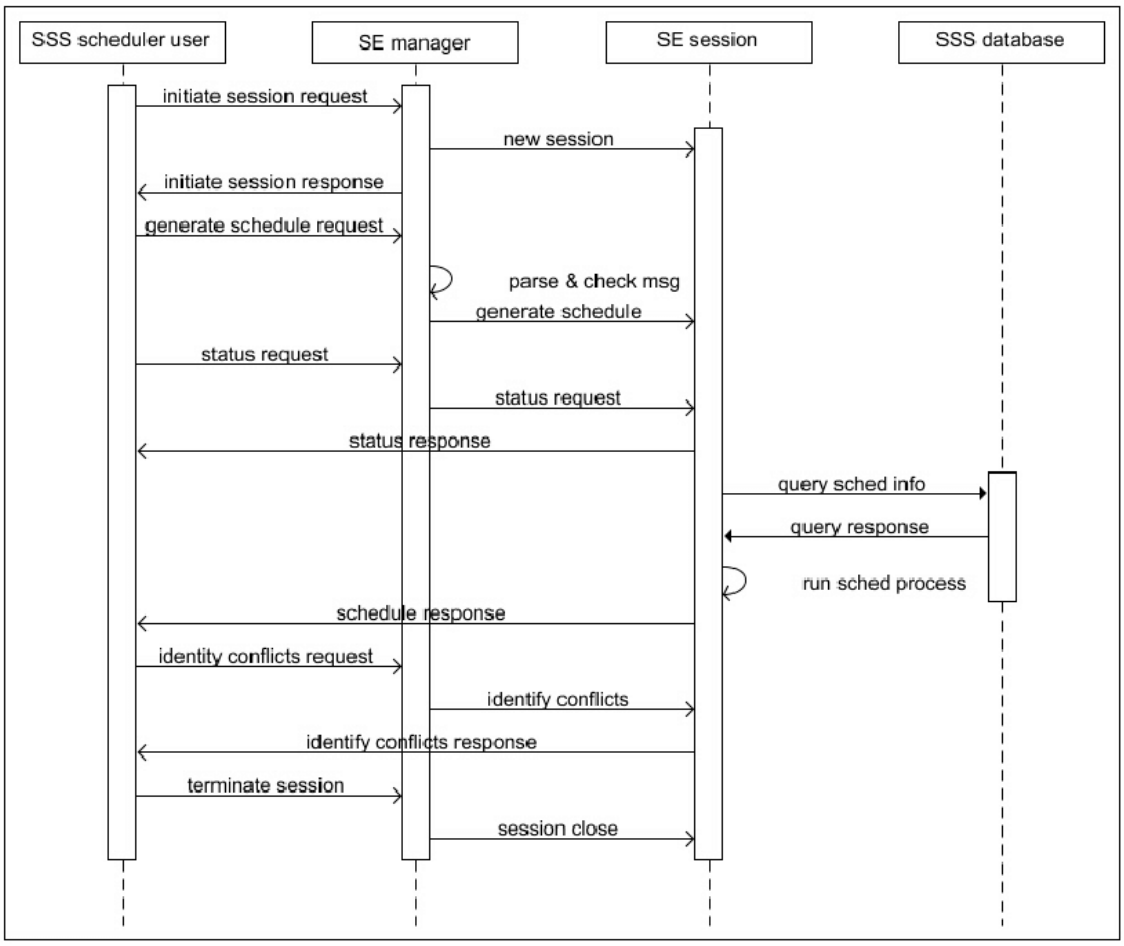



Figure A.2: Sequence diagram for message delivery pipeline.

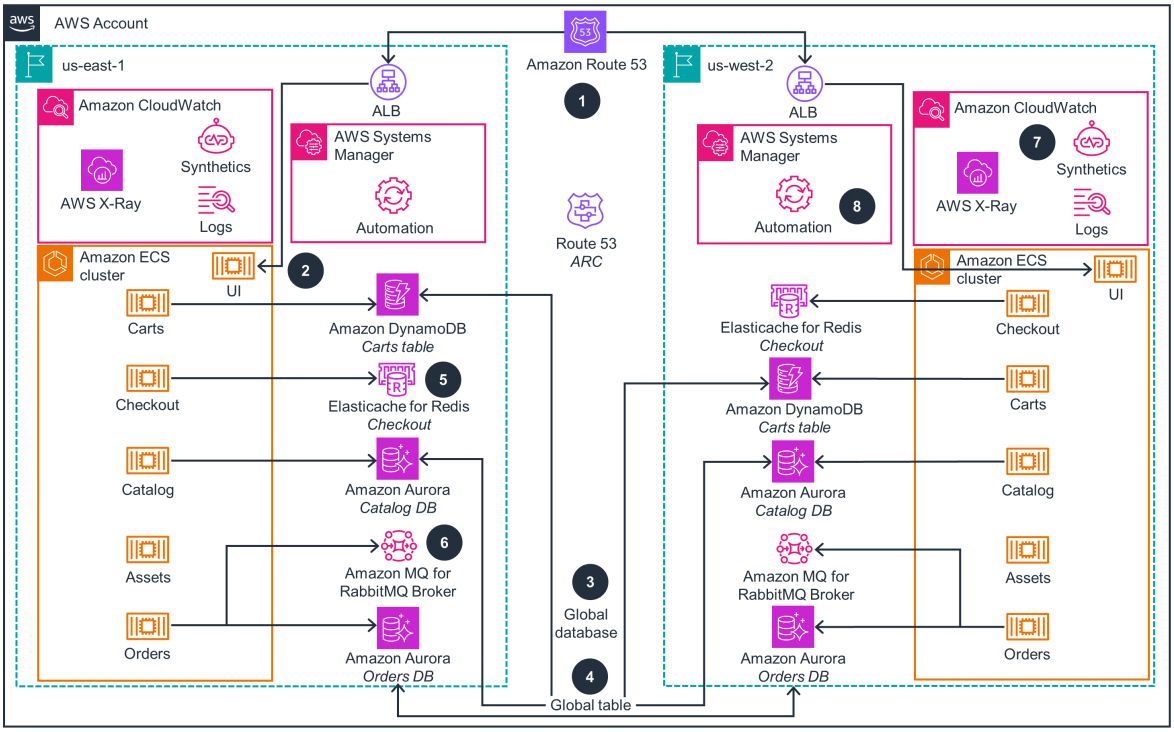



Figure A.3: AWS deployment architecture (detailed view).

## A.4 Selected Code Snippets

Here, you will find practical code examples for key system components, including WebSocket handlers, Redis pub/sub integration, and Nginx proxy configuration.

### A.4.1 WebSocket Message Handler (Node.js)

This snippet demonstrates how the WebSocket server processes incoming messages and delivers them to channel members.

```
ws.on('message', async (raw) => {
  try {
    const frame = JSON.parse(raw);
    if (frame.event === 'msg.send') {
      const saved = await messageService.persist(frame.data);
      await pubsub.publish(saved.channel_id, {
        event: 'msg.deliver',
        data: saved
      });
    }
  } catch (err) {
    logger.warn({ err }, 'invalid frame');
    ws.send(JSON.stringify({
      error: { code: 'BAD_REQUEST', message: 'Invalid JSON frame' }
```

```
    }));
  }
});
```

Listing A.9: WebSocket message handler (simplified)

### A.4.2 Redis Pub/Sub Integration

This example shows how Redis is used for pub/sub messaging, enabling efficient fan-out to multiple clients.

```
const sub = redis.duplicate();
await sub.connect();
await sub.subscribe(`chan:${channelId}`, (msg) => {
  const payload = JSON.parse(msg);
  fanoutService.broadcastToChannel(channelId, payload); // non-blocking
});
```

Listing A.10: Redis Pub/Sub fan-out (sketch)

### A.4.3 Nginx Reverse Proxy for WebSocket

This configuration illustrates how Nginx is set up to proxy WebSocket connections securely and reliably.

```
server {
  listen 443 ssl http2;
  server_name ws.example.com;

  location / {
    proxy_pass http://gateway_pool;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "Upgrade";
    proxy_http_version 1.1;
    proxy_read_timeout 60s;
  }
}
```

Listing A.11: Nginx config snippet for WebSocket proxying

## A.5 Load Testing Scripts

This section provides scripts and configuration files for load testing the messaging system, validating performance and scalability under stress.

### A.5.1 k6 Script (WebSocket Burst)

This script uses k6 to simulate a burst of WebSocket connections and message traffic, testing system limits.

```
import ws from 'k6/ws';
import { check, sleep } from 'k6';

export const options = {
  scenarios: {
    burst: {
      executor: 'constant-vus',
      vus: 500,
      duration: '60s',
    }
  }
};

export default function () {
  const url = 'wss://ws.example.com';
  const params = { headers: { Authorization: `Bearer ${__ENV.TOKEN}` } };
  const res = ws.connect(url, params, function (socket) {
    socket.on('open', () => {
      for (let i = 0; i < 20; i++) {
        socket.send(JSON.stringify({ event: 'msg.send', data: { channel_id:
    'c-1', content: `hello ${i}` }}));
      }
    });
    socket.on('message', (msg) => { /* validate */ });
    socket.setTimeout(function () { socket.close(); }, 5000);
  });
  check(res, { 'status is 101': (r) => r && r.status === 101 });
  sleep(1);
}
```

Listing A.12: k6 WebSocket burst test (example)

### A.5.2 Artillery (Channel Fan-out)

This Artillery configuration is designed to test channel fan-out performance by broadcasting messages to many recipients.

```
config:
  target: "wss://ws.example.com"
  phases:
    - duration: 120
      arrivalRate: 200
  engines:
    ws: {}
scenarios:
  - engine: "ws"
```

```
  flow:
    - send:
        event: "msg.send"
        data:
          channel_id: "c-1"
          content: "broadcast"
    - think: 1
```

Listing A.13: Artillery config for fan-out test (excerpt)

## A.6 Evaluation Data Tables

This section contains tables summarizing latency, throughput, and resource usage metrics collected during system evaluation.

Table A.1: Latency distribution under load (single region, 3 nodes)

| Scenario | p50 (ms) | p95 (ms) | p99 (ms) | Error % |
|---|---|---|---|---|
| 1k conns, 2k msg/s | 38 | 112 | 197 | 0.05 |
| 10k conns, 8k msg/s | 45 | 138 | 265 | 0.12 |
| 30k conns, 15k msg/s | 59 | 171 | 311 | 0.25 |

Table A.2: Throughput vs number of gateway nodes

| Nodes | Msgs/s (avg) | CPU per node (%) | Redis CPU (%) |
|---|---|---|---|
| 1 | 2,900 | 78 | 31 |
| 3 | 8,700 | 72 | 44 |
| 6 | 17,200 | 68 | 59 |

## A.7 Deployment Scripts (Excerpts)

Here, you will find excerpts from deployment scripts, including Dockerfiles, docker-compose setups, and Terraform snippets for cloud infrastructure.

### A.7.1 Dockerfile (Gateway)

This Dockerfile shows how the Node.js gateway service is built and configured for production deployment.

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
ENV NODE_ENV=production
EXPOSE 8080
```

```
CMD ["node", "dist/main.js"]
```

Listing A.14: Dockerfile (Node.js gateway, simplified)

### A.7.2 docker-compose.yml (Local Dev)

This docker-compose file provides a local development environment with all required services, including Redis and PostgreSQL.

```
version: "3.9"
services:
  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: messaging
      POSTGRES_USER: app_user
      POSTGRES_PASSWORD: app_pass
    ports: ["5432:5432"]
  gateway:
    build: .
    environment:
      - REDIS_HOST=redis
      - DB_HOST=db
    ports: ["8080:8080"]
    depends_on: [redis, db]
```

Listing A.15: docker-compose (local development)

### A.7.3 Terraform Snippet (ALB Listener for WSS)

This Terraform snippet demonstrates how to configure an AWS Application Load Balancer listener for secure WebSocket traffic.

```
resource "aws_lb_listener" "wss" {
  load_balancer_arn = aws_lb.main.arn
  port              = "443"
  protocol          = "HTTPS"
  ssl_policy        = "ELBSecurityPolicy-TLS-1-2-2017-01"
  certificate_arn   = var.acm_cert_arn
  default_action {
    type             = "forward"
    target_group_arn = aws_lb_target_group.gateway.arn
  }
}
```

Listing A.16: Terraform (HCL) snippet for ALB listener (pseudo)

## A.8 Experiment Reproducibility Checklist

This section lists the software versions and step-by-step instructions needed to reproduce the experiments and benchmarks described in the thesis.

### A.8.1 Software Versions

This table details the exact software versions used in all experiments to ensure reproducibility and consistency.

Table A.3: Versions used in experiments

| Component | Version |
|---|---|
| Node.js | 20.x LTS |
| Redis | 7.x |
| PostgreSQL | 16.x |
| k6 | 0.50+ |
| Artillery | 2.x |
| Nginx | 1.25+ |

### A.8.2 Steps to Reproduce (Summary)

This summary outlines the main steps required to set up the infrastructure, deploy the system, and run performance tests.

1. Provision infrastructure (VPC, ALB, compute, ElastiCache, DB).
2. Deploy gateway container(s) and set environment variables (no secrets in images).
3. Seed database and create test channels/users.
4. Run k6/Artillery scenarios from a separate load injector VPC/subnet.
5. Collect metrics (CloudWatch, Prometheus, logs) and export CSV for analysis.

## A.9 Additional Logs and Error Samples

This section provides sample logs and error codes to help with troubleshooting and understanding system behavior during failures.

### A.9.1 Structured Log (Masked)

This log example shows the format and key fields of structured logs generated by the gateway service.

```
{
  "ts":"2026-01-17T14:20:01.120Z",
  "level":"info",
  "msg":"delivered",
  "channel":"c-82f5",
```

```
  "message_id":"m-9ad12",
  "latency_ms":42,
  "conn_id":"k3r2",
  "trace_id":"8e4b0a7b-1c6a-4c5c-9f0e-1b2c3d4e5f6a"
}
```

Listing A.17: Gateway structured log (JSON)

### A.9.2 WebSocket Close Codes (Excerpt)

This table lists common WebSocket close codes and their meanings, useful for diagnosing connection issues.

Table A.4: Common WebSocket close codes (excerpt)

| Code | Name | Meaning |
|---|---|---|
| 1000 | Normal Closure | Regular closure without error |
| 1006 | Abnormal Closure | No close frame; network break or peer lost |
| 1008 | Policy Violation | Server closed due to policy (e.g., ACL) |
| 1011 | Internal Error | Unexpected server condition |

## A.10 Glossary of Technical Terms

This glossary defines important technical terms used throughout the appendix and thesis, supporting reader understanding.

- **Backpressure:** Mechanisms to prevent producers from overwhelming consumers by applying flow control.
- **Fan-out:** Broadcasting a message to multiple recipients (e.g., all members of a channel).
- **Presence:** Representation of a user's online/offline/away state and last-seen timestamp.
- **Sharding:** Partitioning data or load across multiple nodes to scale horizontally.
- **Sticky Session:** Routing a client to the same backend node across requests or connections.
- **Trace ID:** Unique identifier propagated across services for distributed tracing.